

UNIVERSIDADE FEDERAL DE SÃO CARLOS

CENTRO DE CIÊNCIAS EXATAS E DE TECNOLOGIA

DEPARTAMENTO DE ENGENHARIA DE PRODUÇÃO



**UM ALGORITMO *ITERATED GREEDY* APLICADO EM *SCHEDULING*
COM REJEIÇÃO E PENALIDADE DE ATRASO**

ALEX DOS SANTOS MARINHO

MONOGRAFIA EM ENGENHARIA DE PRODUÇÃO

UNIVERSIDADE FEDERAL DE SÃO CARLOS
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
Departamento de Engenharia de Produção

ALEX DOS SANTOS MARINHO

UM ALGORITMO *ITERATED GREEDY* APLICADO EM *SCHEDULING*
COM REJEIÇÃO E PENALIDADE DE ATRASO

Monografia apresentado ao Curso de Graduação em Engenharia de Produção da Universidade Federal de São Carlos (UFSCar), Campus São Carlos, como parte dos requisitos para obtenção do título de bacharel em Engenharia de Produção.

Orientador: Prof. Dr. Roberto Fernandes Tavares
Neto

SÃO CARLOS
2015

AGRADECIMENTOS

Agradeço ao meu orientador, Prof. Dr. Roberto Fernandes Tavares Neto, pela dedicação, atenção e ensinamentos, os quais foram essenciais para o desenvolvimento deste trabalho.

À minha família que, com muito amor e orientação, sempre estiveram ao meu lado para que eu buscase um constante desenvolvimento.

À Luiza, pessoa com quem amo partilhar a vida, obrigado pela paciência, pela força e principalmente pelo carinho.

À Universidade Federal de São Carlos (UFSCar), em especial ao Departamento de Engenharia de Produção (DEP) e todos meus professores, por todo conhecimento transmitido durante o curso.

MARINHO, A.S. **Um algoritmo *iterated greedy* aplicado em *scheduling* com rejeição e penalidade de atraso.** 45 f. Monografia (Graduação em Engenharia de Produção) Universidade Federal de São Carlos (UFSCar), São Carlos, 2015.

RESUMO

Em problemas de sequenciamento clássicos, assume-se que todas as tarefas devem ser processadas. No entanto, em muitos casos práticos, principalmente em sistemas de produção *make-to-order*, aceitar todos os pedidos pode causar um atraso na conclusão de ordens, levando a maiores custos de estoque e insatisfação do cliente. Por serem mais semelhantes a problemas reais, o estudo do sequenciamento considerando rejeição tem recebido uma grande atenção de pesquisadores na última década. O presente trabalho se propôs a desenvolver um algoritmo baseado na técnica *Iterated Greedy* e aplicá-lo a problemas de *scheduling* considerando rejeições de tarefas e penalidade de atraso em ambiente de máquina única, comparando os resultados obtidos aos encontrados por outras metaheurísticas na literatura. A partir de modelos matemáticos que simulam o funcionamento de sistemas produtivos, o algoritmo deve decidir quais tarefas devem ou não ser realizadas, definindo um cronograma de modo que todas as restrições do problema sejam obedecidas e o critério de otimização seja minimizado. O critério de otimização é uma combinação linear do somatório dos custos das tarefas processadas (além do custo de processamento, poderá incorrer custos de setup e de atrasos), e do somatório dos custos de rejeição das tarefas não processadas. Os resultados encontrados pelo algoritmo proposto (em termos de resultados e tempos computacionais) foram equivalentes aos resultados obtidos pelas melhores metaheurísticas da literatura. Em relação as 44 instâncias testadas, 36 obtiveram resultados iguais aos ótimos conhecidos na literatura, 2 resultados melhores aos ótimos e nas demais foram obtidos resultados próximos ao ótimo.

Palavras chave: Sequenciamento e Programação de Operações, *Scheduling* com Rejeição, Penalidade de Atraso, Máquina Única, Metaheurísticas, *Iterated Greedy*.

MARINHO, A.S. A iterated greedy algorithm applied in scheduling with rejection and tardiness penalties. 45 p. Monograph (Graduation in Industrial Engineering) - University Federal of São Carlos (UFSCar), São Carlos, 2015.

ABSTRACT

In classical scheduling problems, it is assumed that all jobs are to be processed. However, in many practical cases, especially on make-to-order production systems, accept all requests may cause a delay in the completion of orders, leading to higher inventory costs and customer dissatisfaction. For be more similar to real problems, the study of scheduling with rejection has received great attention by researchers in the past decade. This study aimed to develop an algorithm based on Iterated Greedy technique and apply it in a scheduling problem with rejection and tardiness penalties, in a single machine environment, comparing the results with those found by other metaheuristics in the literature. From mathematical models that simulate the functioning of productive systems, the algorithm must decide what job should or should not be done by setting a schedule so that all constraints of the problem are obeyed and the optimization criterion is minimized. The optimization criterion is a linear combination of the sum of the jobs processed costs (other than processing costs, setup costs and tardiness costs may occur), and the sum of rejection costs of unprocessed jobs. The results found by the algorithm proposed (in terms of results and computational time) were equivalent to the results obtained by the metaheuristics in the literature. Regarding the 44 instances tested, 36 had results equal to optimal solution in the literature, 2 had better than optimal solution and the rest obtained results close to optimal.

Keywords: *Scheduling rejection, Tardiness Penalty, Single-machine, Metaheuristics, Iterated Greedy.*

SUMÁRIO

1 INTRODUÇÃO	08
1.1 CARACTERIZAÇÃO DO TEMA DE PESQUISA	08
1.2 FORMULAÇÃO DO PROBLEMA E OBJETIVO DE PESQUISA	09
1.2.1 Descrição do problema proposto	10
1.2.2 Formulação Matemática	11
1.3 JUSTIFICATIVA DA PESQUISA	13
1.4 ESTRUTURA DO TRABALHO	14
2 REVISÃO DA LITERATURA	15
2.1 <i>SCHEDULING</i>	15
2.2 PROBLEMAS DE <i>SCHEDULING</i>	17
2.3 A OTIMIZAÇÃO ATRAVÉS DA METAHEURÍSTICA <i>ITERATED GREEDY</i>	19
2.4 HEURÍSTICAS APLICADAS EM <i>SCHEDULING</i> COM REJEIÇÃO E PENALIDADE DE ATRASO	21
3 MÉTODO DE PESQUISA	23
3.1 O ALGORITMO <i>ITERATED GREEDY</i> PROPOSTO	25
3.1.1 Inicialização do algoritmo e construção da solução inicial	25
3.1.2 Fase de destruição	26
3.1.3 Fase de construção	27
3.1.3.1 Procedimento inserir	27
3.1.3.2 Procedimento permutar	29
3.1.4 Critérios de aceitação e de parada	30
4 APRESENTAÇÃO E DISCUSSÃO DOS RESULTADOS	31
5 CONCLUSÃO	39
REFERÊNCIAS	40
APÊNDICE A - Fluxograma das etapas do algoritmo desenvolvido	44
APÊNDICE B - Fluxograma das etapas da fase de destruição	45
APÊNDICE C - Fluxograma das etapas do procedimento inserir	46
APÊNDICE D - Fluxograma das etapas do procedimento permutar	47
APÊNDICE E – Interfaces de execução e análise de replicações	48
APÊNDICE F - Representação exemplificativa dos parâmetros de uma instância	49
APÊNDICE G - Representação de uma solução obtida pelo algoritmo proposto	50

APÊNDICE H - Representação exemplificativa das soluções candidatas encontradas durante uma execução.....	51
APÊNDICE I - Representação exemplificativa dos tempos e execuções das sub-rotinas obtidos durante uma execução.....	52

1 INTRODUÇÃO

Neste capítulo o tema objeto de estudo é introduzido. Na Seção 1.1 será exposta a caracterização do tema de pesquisa, onde o problema de sequenciamento de tarefas é contextualizado e descrito resumidamente. A Seção 1.2 apresenta a formulação do problema e objetivos de pesquisa. Nesta Seção também é detalhado o problema que o trabalho se propõe a resolver. A Seção 1.3 traz a justificativa da pesquisa e por fim, a Seção 1.4 apresenta a estrutura remanescente do trabalho.

1.1 CARACTERIZAÇÃO DO TEMA DE PESQUISA

A programação de operações (*scheduling*) é um dos campos de pesquisa mais importantes no contexto da pesquisa operacional e otimização combinatória (LU; NG; ZHANG, 2011). O primeiro estudo voltado para a resolução de um problema de *scheduling* foi realizado por Johnson (1954). Desde então, o assunto vem sendo explorado considerando diversos aspectos e variações.

Para Wight (1984), os dois problemas chave no *scheduling* são: prioridades e capacidade. Ele observa que dos diversos tipos de *scheduling* existente, nas empresas de manufatura, se destaca a programação de operação. O dicionário Apics (2013) define *scheduling* de operação (*operations scheduling*) como a alocação de datas de início e/ou conclusão de operações indicando quando devem ser concluídas caso as ordens de produção sejam realizadas a tempo. Para Sadeh (1994), essencialmente, *scheduling* é definido como a alocação de recursos ao longo do tempo para executar um conjunto de tarefas, atendendo a uma variedade de restrições e objetivos.

Problemas de *scheduling* são especificados em termos de uma classificação de três campos $\alpha \mid \beta \mid \gamma$ conforme o trabalho de Graham et al. (1979), em que α especifica o ambiente de máquina, β as características das tarefas e γ o critério de otimização.

Sempre existiu uma diferença notável entre a teoria e a prática dos problemas de *scheduling*. Ruiz, Serifoglu e Urlings (2008) citam diversos trabalhos que reconhecem esse “gap”. No estudo do *scheduling* clássico, todas as tarefas devem ser processadas, mas na prática, isso não acontece. Para Cesaret, Oğus e Salman (2012), principalmente em sistemas com capacidade de produção limitada e curtos prazos de entrega, deve ser considerada a rejeição de tarefas. Aceitar pedidos sem considerar o impacto sobre a capacidade de produção

pode acarretar sobrecarga nos recursos por alguns períodos, podendo ocorrer atrasos e aumento nos custos.

O primeiro estudo de *scheduling* com rejeição foi publicado por Bartal et al. (2000). A partir de então, o assunto tem recebido uma crescente atenção. Slotnick (2012) e Shabtay, Gaspar e Kaspi (2013) apresentam revisões da literatura sobre o assunto, indicando uma busca pela resolução de problemas cada vez mais próximos da realidade.

1.2 FORMULAÇÃO DO PROBLEMA E OBJETIVO DE PESQUISA

A partir do contexto destacado anteriormente, tem-se uma ideia dos desafios inerentes à programação de operações. A busca de soluções ótimas em problemas de *scheduling* podem ser encontradas através de métodos exatos, contudo, por se tratar de otimização combinatória, para problemas de maior dimensão, sua aplicabilidade se torna limitada, devido à grande demanda de tempo para a análise das soluções, sendo preferível a utilização de métodos heurísticos (DORIGO; STÜTZLE 2004).

Dorigo e Stützle (2004) definem métodos heurísticos, também chamados de métodos aproximados, como sendo o uso de algoritmos para encontrar soluções satisfatórias utilizando um custo relativamente baixo de processamento (pausar métodos exatos após um certo tempo de processamento também é considerado um método aproximado), embora não exista garantia de encontrar a solução ótima. As metaheurísticas, como estrutura algorítmica geral que pode ser aplicada a diferentes problemas de otimização com relativamente poucas modificações, tornaram possível a melhora dos métodos heurísticos.

O seguinte trabalho buscar resolver um problema de *scheduling* em máquina única, considerando rejeição e penalidades, através de uma metaheurística baseada na técnica Iterated Greedy (IG).

Algoritmos IG são baseados em princípios simplificados, de fácil implementação e podem mostrar um excelente desempenho (RUIZ; STUTZLE, 2008).

Desta forma, o problema de pesquisa é:

Uma metaheurística baseada na técnica Iterated Greedy pode ser aplicada em problemas de *scheduling* considerando rejeições de tarefas e penalidade de atraso em ambiente de máquina única, obtendo resultados satisfatórios em comparação com outras metaheurísticas?

Sendo assim, o objetivo principal deste trabalho foi definido como:

Desenvolver um algoritmo baseado na técnica Iterated Greedy e aplicá-lo a problemas de *scheduling* considerando rejeições de tarefas e penalidade de atraso em ambiente

de máquina única, comparando os resultados obtidos aos encontrados por outras metaheurísticas na literatura.

Diante disso, os objetivos específicos do presente trabalho são:

- a) propor um algoritmo baseado em IG para a resolução do problema de *scheduling* em máquina única, de característica *strongly NP-Hard*, segundo Pinedo (2012), representado por $\mathbf{1} \mid r_j, s_{i,j}, \bar{d}_j \mid F_l(\sum f_j(C_j), \sum u_j, \sum c_{i,j})$. O objetivo não é necessariamente minimizar os números de tarefas rejeitadas, mas sim buscar a minimização da combinação linear dos componentes representados em γ (THEVENIN; ZUFFEREY; WIDMER, 2015);
- b) comparar os resultados obtidos com os apresentados por Baptiste e Le Pape (2005) e Thevenin, Zufferey e Widmer (2015), e sugerir possibilidades de melhoria para futuros estudos.

O problema a ser estudado foi desenvolvido por Nuijten et al (2003), representado por um conjunto de instâncias chamado de MaScLib (*Manufacturing Scheduling Library*), inspirado por problemas de *scheduling* industriais encontrados ao longo dos anos pelos autores. Essas instâncias foram criadas para facilitar a pesquisa de *scheduling* na manufatura, fornecendo uma base industrial para testar a programação de algoritmos. Baptiste e Le Pape (2005), resolveram instâncias do problema com até 30 tarefas. Thevenin, Zufferey e Widmer (2015), utilizaram diversas metaheurísticas para resolver o mesmo problema, se destacando a busca tabu, resolvendo satisfatoriamente instâncias com até 500 tarefas.

1.2.1 Descrição do problema proposto

Programar a fabricação de diferentes tipos de produtos, usando um conjunto compartilhado de recursos, pode ser uma tarefa difícil. Um bom sequenciamento além de satisfazer restrições temporais, deve considerar restrições de atribuições de recursos, limitações de capacidade e de configuração. Thevenin, Zufferey e Widmer (2015), citam possíveis situações realistas em que o modelo se encaixa. Por exemplo, uma empresa que produz embalagens plásticas e quer programar sua produção. Dependendo dos produtos a serem processados consecutivamente, diferentes operações devem ser executadas: limpeza, troca do corante das cores do plástico, alteração no molde da injetora, por exemplo. Tais operações possuem tempo e custos consideráveis e devem ser minimizados.

Processos sequenciais de produtos da mesma família não precisam de quaisquer configurações.

É adotada a estratégia *Make to Order*, ou seja, nada deve ser produzido ou comprado antes da hora exata, as ordens são geradas de acordo com os pedidos e não podem ser processadas antes da entrega das matérias primas associadas.

Se a capacidade de produção está sobrecarregada, a empresa tende a atrasar a entrega. Atrasos geram insatisfação nos clientes. Quando não existir capacidade disponível de processamento, em vez de atrasar muito uma entrega, a empresa pode subcontratar serviços, honrando os pedidos, ou simplesmente desconsidera-los. No entanto, rejeitar trabalhos irá implicar em penalidades por rejeição.

1.2.2 Formulação matemática

A formulação matemática do problema proposto foi extraída de Thevenin, Zufferey e Widmer (2015), e classificada conforme a notação de Graham et al. (1979).

Para cada tarefa j , foram dados os seguintes parâmetros:

- a) p_j : tempo de processamento da tarefa j ;
- b) r_j : data de liberação da tarefa j . É o momento em que é possível iniciar o processamento da tarefa j . Em sistemas de manufatura, pode ser considerado a data em que a matéria prima é entregue;
- c) d_j : data de entrega da tarefa j . Representa a data em que a satisfação do cliente começa a decair (é representada por uma função crescente de custo $f_j(C_j)$). Pode ser definida como a data em que $f_j(.)$ começa a ser maior que zero;
- d) $\bar{d}_j$: data limite da tarefa j . Agendar uma tarefa depois desta data não é possível. Pode ser o momento em que custo da penalidade de agendamento se torna mais elevado que o custo de abandonar a tarefa, ou o momento que o cliente não deseja mais o produto. Assim, cada tarefa j deve ser realizada dentro de uma janela de tempo $[r_j, \bar{d}_j]$;
- e) u_j : custo de abandono da tarefa j , sendo a penalidade aplicada caso a tarefa j não seja executada;
- f) s_{ij} : tempo de *setup*. As tarefas pertencem a diferentes famílias, que correspondem a diferentes tipos de produtos. Quando a máquina processar em sequência duas tarefas i e j de diferentes famílias, implicará num tempo de *setup*;
- g) c_{ij} : custo de *setup*. Pode ser considerado o custo do pagamento de empregados que preparam o *setup* máquina e custos de materiais adicionais.

As variáveis de decisão são definidas como:

- a) t_j : tempo inicial da tarefa j ;
- b) C_j : tempo de conclusão da tarefa j .

$$x_{jk} = \begin{cases} 1, & \text{se a tarefa } j \text{ preceder a tarefa } k \\ 0, & \text{caso contrário} \end{cases}$$

$$z_j = \begin{cases} 1, & \text{se a tarefa } j \text{ não for realizada} \\ 0, & \text{caso contrário} \end{cases}$$

Foi adicionado artificialmente uma última tarefa $n+1$, sendo:

$$p_{n+1} = 0, \quad s_{(j)(n+1)} = 0, \quad c_{(j)(n+1)} = 0, \quad \forall j.$$

$$\min \left[\sum_{j=0}^n \sum_{k=1}^{n+1} x_{jk} c_{jk} \sum_{j=1}^n [f_j(C_j) + z_j (u_j - f_{jk}(r_j))] \right] \quad (1.1)$$

Sujeito a

$$C_j = t_j + p_j \quad \forall j \quad (1.2)$$

$$t_j \geq C_k + s_{kj} x_{kj} + (x_{kj} - 1) \bar{d}_k \quad \forall j, k \quad (1.3)$$

$$t_j \geq r_j \text{ e } C_j \leq \bar{d}_j + z_j (r_j - \bar{d}_j) \quad \forall j \quad (1.4)$$

$$z_j + \sum_{k=1}^{n+1} x_{jk} = 1 \quad \forall j \neq n+1 \quad (1.5)$$

$$z_j + \sum_{k=0}^n x_{jk} = 1 \quad \forall j \neq 0 \quad (1.6)$$

$$\sum_{j=0}^{n+1} x_{j0} = 0 \quad \text{e} \quad \sum_{j=0}^{n+1} x_{(n+1)j} = 0 \quad \text{e} \quad t_0 = 0 \quad (1.7)$$

$$x_{jk} \in \{0,1\} \quad z_j \in \{0,1\} \quad \forall j \quad (1.8)$$

A função objetivo (1.1) expressa o critério de desempenho procurado, através da minimização do atraso ponderado, formado pelo somatório dos custos de *setup*, considerando o custo $f_j(C_j)$ (custo de completar a tarefa j_i no tempo C_i) se a tarefa for realizada, e em caso contrário, uma penalidade u_j . O tempo de conclusão para cada tarefa não realizada é virtualmente ajustado para a sua data de liberação r_j , pela equação 1.4. Por essa razão, na

Equação 1.1, para cada tarefa não realizada, foi adicionado um custo u_j e removido o custo $f_j(C_j)$ associado a data de liberação.

A Equação 1.2 descreve o tempo de conclusão da tarefa j .

A restrição 1.3 é usada para calcular o tempo de início de cada tarefa. Se j sucede k , então j deve iniciar depois do final de k somado ao tempo de *setup* requerido, caso contrário $(x_{kj} - 1)$ é igual a -1 , e a restrição resultante é menos restritiva que $C_j \geq 0$, assim como $\bar{d}_j \geq C_j$.

A restrição 1.4 impõe que cada tarefa a ser realizada seja agendada dentro de sua janela de tempo e caso o tempo de conclusão seja menor ou igual a data de liberação, a tarefa não será realizada.

As restrições 1.5 e 1.6 especificam que cada tarefa deve ter um sucessor e um antecessor (ou será rejeitada).

A restrição 1.7 restringe a tarefa 0 a não ter antecessores, a tarefa $n+1$ a não ter sucessores, e define como zero o tempo inicial da programação.

1.3 JUSTIFICATIVA DA PESQUISA

Para Fernandes e Godinho Filho (2010, p. 191) “a manufatura moderna geralmente requer distinção e ou diversificação de produtos, produção em pequenos lotes, rápida entrega e aderência a datas de entrega prometidas”. Pinedo (2012) afirma que a aplicação de técnicas de *scheduling* permite o desenvolvimento de diferenciais competitivos, através da diminuição de custos, flexibilidade de produção aliados a menores tempos de entrega não só na maioria dos sistemas de manufatura e produção, sendo importante também em configurações de transportes e distribuição, bem como nas empresas prestadoras de serviços.

A necessidade das empresas de tornarem seus processos produtivos mais eficientes estimula uma crescente tendência em solucionar problemas de *scheduling* que representem cada vez mais os problemas reais, considerando a possibilidade de rejeição de tarefas devido à limitação de recursos.

Este trabalho segue essa tendência, buscando uma metaheurística que de forma simplificada e eficiente encontre soluções satisfatórias em problemas complexos utilizando baixo processamento computacional.

1.4 ESTRUTURA DO TRABALHO

Este trabalho será dividido em cinco capítulos. No Capítulo 1, é feita a introdução, expondo a caracterização do tema da pesquisa, os objetivos e justificativas da pesquisa, assim como a apresentação do problema considerado neste trabalho e suas características. O Capítulo 2 traz uma revisão bibliográfica sobre problemas de sequenciamento na literatura, apresentando um breve histórico do *scheduling* e suas principais características. Neste capítulo também é apresentado a metaheurística *Iterated Greedy* “pura”, e detalhada a sua estrutura. O Capítulo 3 apresenta detalhadamente a metodologia que será utilizada nesta pesquisa, incluindo a descrição da metaheurística baseada em *Iterated Greedy* e das demais heurísticas trazidas pela literatura para resolver o problema proposto. O Capítulo 4 traz os resultados obtidos e comparações com outras metaheurísticas trazidas pela literatura. No Capítulo 5 será exposta a conclusão obtida a partir dos dados analisados e as considerações finais.

2 REVISÃO DE LITERATURA

Neste capítulo serão apresentados os principais aspectos sobre o *scheduling*, um levantamento histórico demonstrando a sua importância e a variedade de abordagens tomadas para melhorá-lo.

Também serão demonstradas as principais características dos problemas de *scheduling*, suas notações e classificações. Ao final do capítulo é descrito a estrutura e o funcionamento da metaheurística *Iterated Greedy* e explicadas as demais heurísticas trazidas pela literatura para resolver problemas de *scheduling* com rejeição e penalidade de atraso.

2.1 SCHEDULING

Compreender as maneiras com que a programação de operações tem sido realizada ao longo dos anos é fundamental para a análise e aprimoramento das existentes técnicas de *scheduling*. De acordo com Herrmann (2006a), o *scheduling* teve seu início de forma simplificada, com a listagem apenas do início e término dos trabalhos ou ordens, sem serem fornecidas informações referentes aos trabalhos, como tempo de processamento ou tempo exigido para as operações individuais. Este tipo de *scheduling* foi amplamente utilizado antes do surgimento de métodos formais.

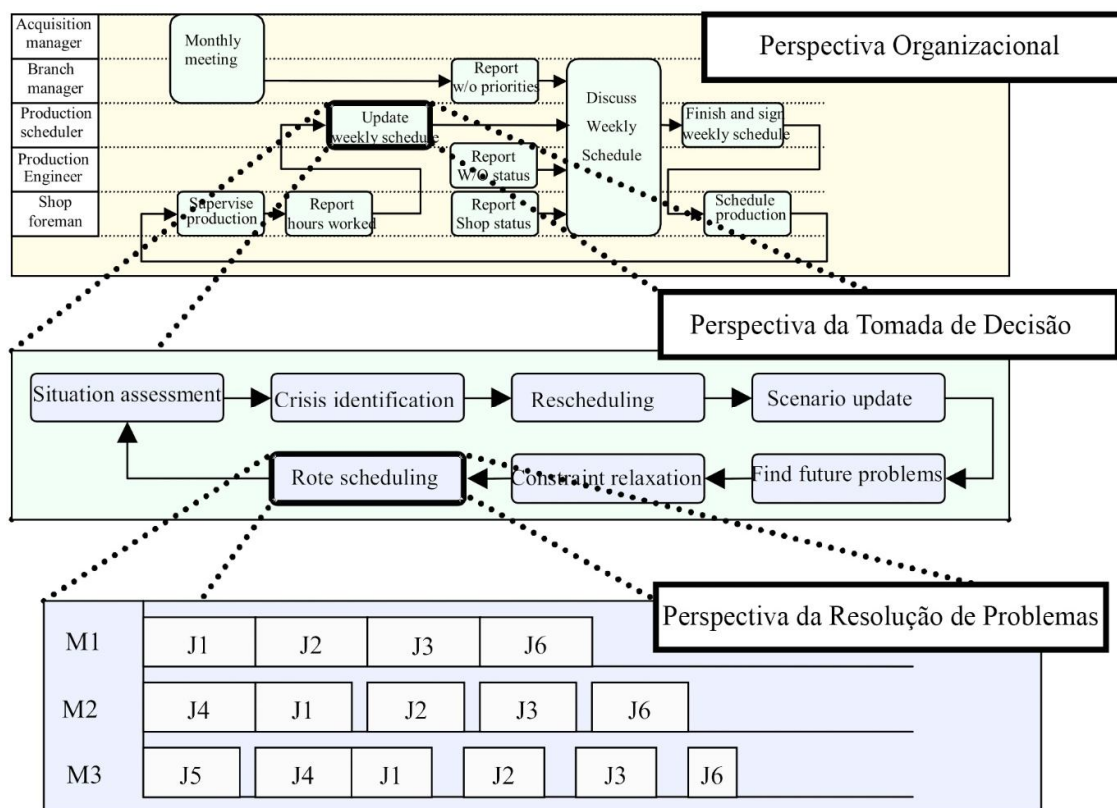
Frederick Taylor separou a programação de operações como uma função distinta da gestão da produção, iniciando o uso de métodos formais. Com isso, grandes mudanças do *scheduling* vieram a ocorrer. A primeira ocorreu com as criações de Henry Gantt para compreender as complexas relações entre os homens, máquina, ordens e tempo. Gantt foi o pioneiro em desenvolver gráficos para visualizar a programação da produção, inovando ao usar o tempo como uma forma de medir tarefas e barras horizontais para representar as partes produzidas e registrar tempos de trabalho (HERRMAN, 2010).

O primeiro a tratar o *scheduling* como um problema de otimização foi Johnson (1954). Neste trabalho, Johnson analisou as propriedades da solução ótima e apresentou um algoritmo para construir tal solução. Estabelecendo uma padronização para a análise de problemas de *scheduling*, este trabalho se tornou uma grande influência, sendo provavelmente o mais citado no campo do *scheduling* (HERRMAN, 2010).

Os trabalhos históricos de Taylor, Gantt e Johnson mostraram a importância distinta das três perspectivas do *scheduling*: Organizacional, tomada de decisão e resolução de problemas. (HERRMANN, 2010).

A partir das mudanças na organização originadas por Taylor, Gantt desenvolveu diagramas para aprimorar o processo de tomada de decisão, e finalmente Johnson estudou a otimização de problemas de *scheduling*. Estas três perspectivas estão relacionadas, ilustrada pela Figura 2.1.

Figura 2.1 Perspectivas do *scheduling* na produção



Fonte: Adaptado de Herrmann (2010)

A perspectiva organizacional leva ao melhor entendimento do fluxo de informações, disponibilizando informações atualizadas e precisas para a melhor tomada de decisão, permitindo o compartilhamento de resultados entre as pessoas certas no momento certo (MACCARTHY, 2006). A perspectiva da tomada de decisão descreve as configurações diárias das programações, sendo essencial para evitar futuros problemas e auxiliar na resolução dos atuais. Na camada de resolução de problemas, heurísticas executam cálculos sobre os dados para gerar e avaliar sequenciamentos otimizados. A perspectiva da resolução de problemas leva ao desenvolvimento de melhores modelos matemáticos e algoritmos que geram programações mais eficientes. A capacidade de formular um problema de forma rigorosa e analisa-lo para encontrar meios de otimização tem atraído muitos pesquisadores.

As três perspectivas sugerem que existem diversos caminhos para aperfeiçoamento do *scheduling*, as abordagens se complementam entre si e podem ser combinadas com uma estratégia integradora que começa adotando a perspectiva organizacional e termina considerando a perspectiva de resolução de problemas (HERRMANN, 2006b).

2.2 PROBLEMAS DE *SCHEDULING*

A partir do estudo Johnson (1954), os problemas de *scheduling* se tornaram mais sofisticados, cujas resoluções exigiam uma nova abordagem diferenciada. Na década de 1970, vários problemas de *scheduling* considerados NP-difícil passaram anos sem solução, já que os algoritmos exatos da época eram incapazes de resolvê-los. Segundo Herrmann (2006a) “o poder esmagador” da tecnologia da informação para coletar, visualizar, processar e compartilhar dados de forma simples e rápida tornou possível o avanço das pesquisas em métodos de otimização, tendo como resultado algoritmos poderosos aplicados com sucesso em diversos problemas complexos de *scheduling*.

Conforme definido por Baker (2009), o *scheduling* está preocupado com o problema da alocação de recursos escassos para atividades ao longo do tempo. Problemas de *scheduling* são, em geral, não triviais, complexos e com diversas restrições a serem consideradas. Para alguns problemas, não existe qualquer algoritmo capaz de achar uma solução ótima em tempo satisfatório.

Os métodos exatos garantem a solução ótima desde que tenha tempo e espaço ilimitados. Podem ser utilizadas técnicas além da simples análise do conjunto completo das possíveis soluções visando a otimização desses algoritmos.

Quando impossíveis de resolver deterministicamente, necessitam de técnicas que encontrem soluções satisfatórias. Heurísticas são métodos que encontram rapidamente soluções satisfatórias com certa qualidade. As metaheurísticas são abordagens de nível superior, que podem combinar métodos e serem facilmente aplicados em diferentes problemas distintos (ZUFFEREY, 2012).

Os problemas de *scheduling* podem ser descritos da seguinte maneira, conforme Pinedo (2012):

- a) n tarefas $\{j_1, j_2, \dots, j_n\}$ devem ser processadas por m máquinas $\{i_1, i_2, \dots, i_m\}$.
Usualmente, o subscrito j refere-se a uma tarefa e i refere-se a uma máquina;
- b) todos os problemas de *scheduling* consideram n e i finitos;

- c) o processamento de uma tarefa j em uma máquina i é chamado de operação com duração p_{ij} conhecida. O subscrito i é omitido caso o tempo de processamento da tarefa j não dependa da variação de máquinas, ou em problemas de máquina única;
- d) o instante que a tarefa j está disponível para processamento é conhecido como r_j (data da liberação da tarefa);
- e) a data de entrega d_j de uma tarefa j representa a data prometida de entrega ou data de conclusão (pode ser considerada a data prometida no momento da venda para o cliente). Realizar uma tarefa após d_j é permitido, mas uma penalidade ocorrerá. Quando uma data de entrega deve ser cumprida rigorosamente, é denominada como a data limite (*deadline*) e representada como $\bar{d}_j$;
- f) o peso w_j de um trabalho j é basicamente um fator de prioridade, representando a importância de uma tarefa j em relação as outras do sistema.

Nesse contexto, programar a produção consiste em definir a ordem de entrada das tarefas a serem executadas em cada máquina. Assim, a programação da produção tem como um de seus objetivos otimizar as medidas de desempenho. Para caracterizar problemas de máquina única, Baker (2009) utiliza as seguintes suposições:

- a) no tempo inicial, todas as tarefas n estão disponíveis para processamento;
- b) a máquina pode processar apenas uma tarefa por vez;
- c) tempos de setup são independentes da programação e estão incluídos nos tempos de processamento;
- d) as características das tarefas são determinísticas e conhecidas antecipadamente
- e) a máquina está sempre disponível e nunca quebra;
- f) cada tarefa deve ser processada até seu término.

Conforme Graham et al. (1979), um problema de *scheduling* pode ser descrito através da tripla $\alpha | \beta | \gamma$. O campo α , referente ao ambiente de máquina, pode conter apenas uma entrada. O campo β detalha as características do processo e as restrições, pode estar vazio, ou contar uma ou mais entradas. Já o campo γ descreve qual o objetivo a ser minimizado e geralmente contém apenas uma entrada.

Segundo Pinedo (2012), os possíveis ambientes de máquinas especificadas no campo α são:

- a) máquina única (1) – O caso da máquina única é o mais simples dos possíveis ambientes de máquinas, onde existe uma única máquina disponível no sistema.

Pode ser visto como um caso especial dentre todos os outros ambientes de máquina mais complexos;

- b) máquinas idênticas em paralelo (P_m) – Há m máquinas idênticas em paralelo. Uma tarefa j requer uma única operação e pode ser processada em qualquer uma das m máquinas ou em qualquer que pertença a um dado subconjunto de máquinas;
- c) máquinas com velocidades diferentes em paralelo (Q_m) – Há m máquinas em paralelo com velocidades de processamento diferentes;
- d) máquinas em paralelo não relacionadas (R_m) – um conjunto de máquinas diferentes em paralelo;
- e) *flowshop* (F_m) – Em um *flowshop* com m máquinas, todas estão em série, e os elementos a serem processados seguem o mesmo roteiro de fabricação;
- f) *flowshop* flexível (FF_c): O *flowshop* flexível é uma generalização do *flowshop* e dos ambientes de máquinas idênticas em paralelo. Em vez de máquinas em série, existem c estágios em série com máquinas idênticas em paralelo. As tarefas são executadas primeiro no estágio 1, em seguida no estágio 2, e assim por diante;
- g) *jobshop* (J_m): no *Jobshop* com m máquinas cada tarefa tem uma rota pré-determinada a seguir. Pode visitar mais de uma vez algumas máquinas e nenhuma outras, podendo o roteiro de fabricação ser distinto entre os elementos a serem processados;
- h) *jobshop* flexível (FJ_c): é uma generalização do *Jobshop* e dos ambientes de máquinas em paralelo. Em vez de máquinas em série, existem c estágios em série, cada estágio possui um número de máquinas idênticas em paralelo;
- i) *openshop* (O_m): No *Openshop* não existe uma ordem de fabricação. A ordem de execução de determinada tarefa é livre.

2.3 A OTIMIZAÇÃO ATRAVÉS DA METAHEURÍSTICA *ITERATED GREEDY* APLICADO EM MÁQUINA ÚNICA

A metaheurística IG proposta por Ruiz e Stützle (2005) funciona através da aplicação de duas fases de forma iterativa. Na fase de destruição, eliminam-se algumas tarefas da solução corrente, obtendo uma solução parcial, e na fase de construção, as tarefas são inseridas nesta solução parcial até formar uma nova solução completa, também conhecida como solução corrente.

O Algoritmo 2.1 a seguir, retirado de Ruiz e Stützle (2006) ilustra o funcionamento do IG aplicado em *scheduling* clássico:

Algoritmo 2.1 A estrutura geral do algoritmo IG

Entrada: <i>critérioParada</i>	
1	início
2	$s_0 := \text{GerarSoluçãoInicial};$
3	$s := \text{BuscaLocal}(s_0);$ %opcional
4	repita
5	$s_p := \text{Destruição}(s);$
6	$s' := \text{Construção}(s_p);$
7	$s' := \text{BuscaLocal}(s');$ %opcional
8	$s := \text{CritérioAceitação}(s, s');$
9	até <i>critérioParada</i> ser satisfeito;
10	retorna $s;$
11	fim

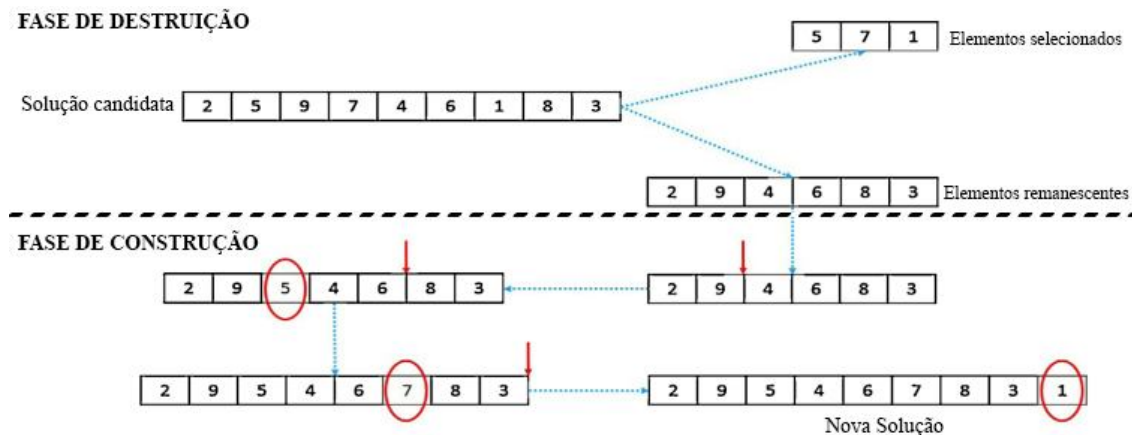
Fonte: Ruiz e Stützle (2006).

Na linha 2 é gerada uma solução inicial de maneira aleatória, podendo esta ser refinada por um método de busca local (linha 3). A seguir o Algoritmo entra em um laço de repetição até que o critério de parada seja satisfeito. Dentro deste laço, a solução sofre um processo de destruição e construção, podendo novamente ser aplicado um método de busca local para refinamento da solução corrente. O critério de aceitação (linha 8) define se a solução corrente será destruída no próximo ciclo ou a solução candidata. A solução candidata é a melhor solução obtida entre todas as iterações em uma execução do algoritmo.

Para Pinedo (2012), modelos de ambientes de máquina única são importantes por várias razões. Além de serem simples, podem ser considerados casos especiais de todos os outros ambientes. Esses modelos muitas vezes possuem propriedades que nem máquinas em paralelo nem máquinas em série têm. Os resultados que podem ser obtidos por modelos de máquina única não só podem oferecer *insights* sobre o ambiente de máquina única, mas também fornece uma base heurística que são aplicáveis a ambientes mais complexos. Na prática, os problemas de programação em ambientes de máquinas mais complexas muitas vezes são decompostos em subproblemas que lidam com máquinas únicas, como por exemplo em algum ambiente mais complexo, onde o gargalo pode dar origem a um modelo de máquina única.

Na Figura 2.2 a seguir, temos um exemplo numérico do algoritmo *Iterated Greedy* aplicado em máquina única.

Figura 2.2 Fases de destruição e construção da metaheurística *Iterated Greedy*



Fonte: Adaptado de Abdollahpour e Rezaeian (2015).

2.4 HEURÍSTICAS APLICADAS EM *SCHEDULING* COM REJEIÇÃO E PENALIDADE DE ATRASO

Thevenin, Zufferey e Widmer (2015) desenvolveram 6 heurísticas para resolver problemas de *scheduling* com rejeição e penalidade de atraso, detalhadas a seguir:

- a) a heurística construtiva *Greedy* insere uma a uma tarefas rejeitadas escolhidas aleatoriamente na sequência de tarefas processadas, em uma posição de menor custo. As tarefas processadas deslocadas, caso quebrem qualquer restrição do problema, serão rejeitadas;
- b) o algoritmo *Descent*, uma metaheurística de busca local, a partir de uma solução inicial pré-estabelecida (através da heurística *Greedy*), explora toda a vizinhança de soluções em busca de uma solução de menor custo, utilizando procedimentos de remoção de tarefas, adição, reposicionamento e trocas de posição entre tarefas processadas;
- c) a metaheurística Tabu incrementa ao algoritmo *Descent* a utilização de listas restringindo alguns procedimentos de serem executados em tarefas recém movimentadas durante um intervalo de iterações definido através de parâmetros de inicialização;
- d) a metaheurística TabuDiv aprimora o algoritmo Tabu, através da implementação de restrições de vizinhança e mecanismos de diversificação baseados na remoção de tarefas processadas “antigas” da sequência. Define-se uma idade para as tarefas processadas, a partir do número de iterações desde sua inserção na

sequência. 30% das tarefas mais antigas são rejeitadas a cada 500 iterações. O algoritmo TabuDiv também possui procedimentos para otimizar instâncias que possuem tarefas idênticas;

- e) o algoritmo Evolucionário AMA^{Mem} , baseado na evolução natural, escolhe as melhores soluções candidatas para gerar novas soluções através de procedimentos de recombinação e/ou mutação. A recombinação é aplicada a duas ou mais soluções candidatas selecionadas (os chamados pais), gerando novas soluções (as crianças). A mutação é aplicada a uma única solução candidata e resulta em uma nova solução. Execuções de recombinação e mutação levam a um conjunto de novas soluções que competem com as antigas para criar uma próxima geração de soluções. São utilizados mecanismos de intensificação (baseados no algoritmo tabu) e de diversificação;
- f) o algoritmo AMA^{SP} é idêntico ao AMA^{Mem} , com algumas modificações no procedimento de recombinação.

3 MÉTODO DE PESQUISA

A pesquisa em questão possui caráter quantitativo, possibilitando ao pesquisador trabalhar com as reações, mensuração de fatores, hábitos e atitudes a partir da temática estudada, por meio de sua amostra que irá representar estatisticamente. As características principais da pesquisa quantitativa utilizadas neste trabalho foram detalhadas por Terence e Escrivão Filho (2006), sendo elas:

- a) segue um plano pré-estabelecido, com o intuito de enumerar ou medir fatores;
- b) se embasa em teorias para desenvolver as hipóteses e as variáveis da pesquisa;
- c) examina as relações entre as variáveis por métodos experimentais ou semi-experimentais, controlados com rigor;
- d) utiliza em grande parte das vezes instrumento estatístico para a análise de seus dados;
- e) privilegiam estudos do tipo “antes e depois”, propiciando análises estáticas e instantâneas da realidade como se fossem fotografias.

Segundo Miguel (2012), a pesquisa em gestão de produção e operações baseada em modelos quantitativos parte da premissa de que é possível construir modelos que expliquem pelo menos parte dos problemas de tomada de decisão encontrados em problemas reais. O trabalho em questão utilizou o método de pesquisa quantitativa Modelagem/Simulação, que segundo Martins (2012), tem-se uma manipulação de variáveis em um modelo abstrato, sem contato com a realidade. Este contato com a realidade se encontra apenas no estabelecimento do modelo a ser utilizado, não em sua análise ou resolução. Em grande parte das vezes, técnicas computacionais são utilizadas para simular o funcionamento de sistemas produtivos, a partir de modelos matemáticos (BERTO; NAKANO, 2000).

Conforme Bertrand e Fransoo (2002), em relação à pesquisa quantitativa, utilizando-se do método de Modelagem e Simulação, podemos encontrar duas abordagens, sendo elas a abordagem empírica e a axiomática.

Nas pesquisas axiomáticas o objetivo seria a obtenção de soluções dentro de uma modelagem já definida. Assim, produziria novos conhecimentos sobre o comportamento de variáveis do modelo a ser estudado. Para Morabito e Pureza (2012), a resolução do modelo é uma das fases centrais da pesquisa axiomática, enquanto que na pesquisa empírica a preocupação primária do pesquisador é assegurar que existe um modelo ajustado entre as observações e as ações da realidade (BERTRAND; FRANSOO, 2002).

Dentro da abordagem axiomática, Morabito e Pureza (2012) citam a pesquisa axiomática normativa, baseada em modelos que prescrevem uma decisão para um problema, em geral, modelos de programação matemática. Para Bertrand e Fransoo (2002), para a abordagem axiomática normativa o objetivo principal é a resolução do modelo. “A pesquisa axiomática desenvolve normas, políticas, estratégias e ações, a fim de melhorar os resultados disponíveis na literatura, encontrar uma solução ótima para um problema novo ou comparar desempenho de estratégias que tratam um mesmo problema” (MORABITO; PUREZA, 2012).

No presente estudo a investigação a ser feita em busca dos objetivos traçados será realizada com base em metodologia quantitativa, com o uso do método de simulação, com abordagem axiomática normativa, permitindo assim, analisar os modelos propostos e possibilitar traçar os melhores meios de solução para os fatores utilizados.

Com isto, para a realização desta pesquisa, estão sendo realizados os seguintes passos:

- Passo 1** Estudar o algoritmo *Iterated Greedy*, suas principais variações e estratégias de implementação;
- Passo 2** Estudar o problema de *scheduling* utilizando experimentos das instâncias do *Manufacturing Scheduling Library* gerados por Nuijten et al 2003, propor e implementar um algoritmo IG com variações que permita resolvê-las;
- Passo 3** Analisar os resultados encontrados e compará-los com os resultados encontrados por Baptiste e Le Pape (2005) e Thevenin, Zufferey e Widmer (2015);
- Passo 4** Elaborar conclusões a partir da análise dos dados e possivelmente propor estudos futuros.

3.1 O ALGORITMO *ITERATED GREEDY* PROPOSTO

A seguir será detalhado o funcionamento da metaheurística desenvolvida para a resolução do problema proposto. A estrutura base do algoritmo IG foi expandida para solucionar problemas de *scheduling* com rejeição, através de implementação de procedimentos baseados na metaheurística NEH, desenvolvida por Nawaz (1983). O Algoritmo 3.1 a seguir ilustra a estrutura geral do algoritmo proposto. Todos os passos previstos no Algoritmo 2.1 são contemplados neste Algoritmo 3.1, suas etapas descritas no APÊNDICE A e as principais diferenças detalhadas nas Seções a seguir.

Algoritmo 3.1 A estrutura geral do algoritmo desenvolvido baseado em IG

Entrada: *critérioParada (Tempo e Função Objetivo)*

```

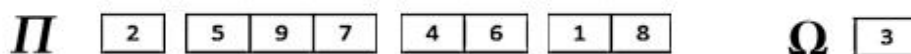
1  Inicialização do algoritmo
2  | Carregue os parâmetros da instância para as estruturas de dados na memória do algoritmo
3  Fim da inicialização
4  Defina uma solução inicial  $s$ , sendo  $\Pi = \phi$  e  $\Omega =$  todas as tarefas da instância
5   $s_c = s$ 
6  repita Neste nível, cada execução é chamada iteração
7  |  $s :=$  Destruir( $s$ );
8  |  $s :=$  Inserir( $s$ );
9  |  $s :=$  Permutar ( $s$ );    %opcional
10 | se  $s < s_c$  então
11 | |  $s_c = s$ 
12 | fim
13 |  $s :=$  CritérioAceitação ( $s, s_c$ );
14 até critérioParada ser satisfeito;
```

Fonte: Próprio autor.

3.1.1 Inicialização do algoritmo e construção da solução inicial

Uma solução do problema proposto é representada por uma dupla (Π, Ω) . Nesta representação, Π representa a sequência de tarefas realizadas, enquanto Ω indica as tarefas rejeitadas.

Figura 3.1 Representação da solução

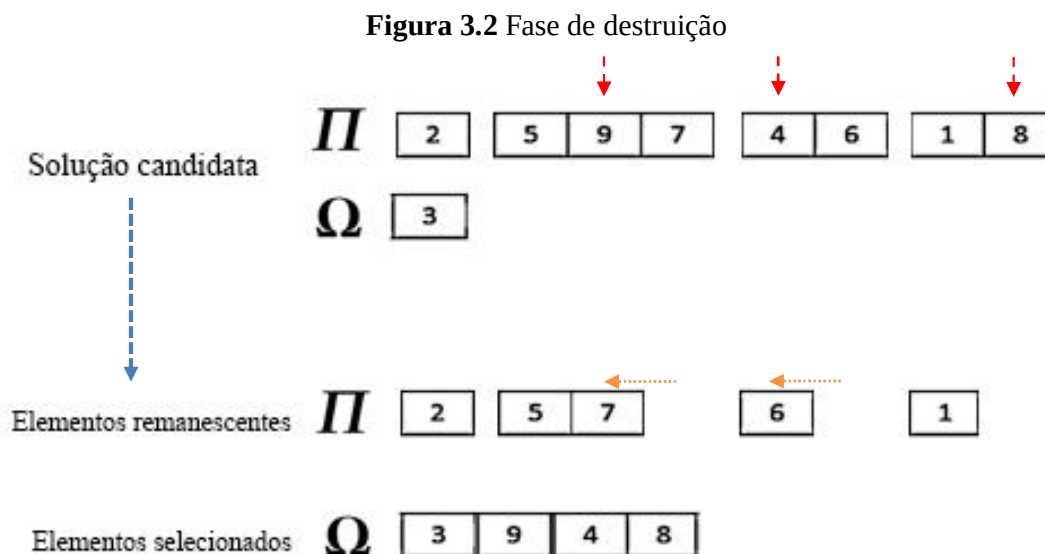


Fonte: Próprio autor.

Ao iniciar o algoritmo, todas as tarefas da instância serão inseridas no conjunto Ω , gerando a solução inicial. Na primeira iteração, sendo o conjunto Π vazio, o algoritmo desconsidera a destruição, seguindo para o procedimento inserir descrito no Algoritmo 3.3, que determinará a solução corrente adicionando um a um, elementos em Π até a formação de uma sequência completa.

3.1.2 Fase de destruição

Na fase de destruição, a partir da solução corrente, o algoritmo seleciona aleatoriamente elementos de Π e os insere em Ω . A quantidade de elementos a serem selecionados d é definida pelo usuário antes da execução do algoritmo. Na imagem exemplificativa a seguir, a partir do parâmetro $d = 3$, três elementos (9, 4 e 8) foram escolhidos aleatoriamente e inseridos em Ω .



Fonte: Próprio autor.

Ao retirar uma tarefa da sequência em Π , o algoritmo irá deslocar os elementos a direita para serem processados o mais cedo possível. A modelagem computacional da fase destruir é ilustrada no Algoritmo 3.2, e suas sub-rotinas detalhadas no APÊNDICE B.

Algoritmo 3.2 A estrutura geral da sub-rotina destruir

destruir (s)	
1	inicialização do procedimento
2	tarefasDestruídas = mínimo (parâmetro d, tarefas no conjunto Π)
3	fim da inicialização
4	se tarefasdestruídas > 0 então
5	para cada tarefasdestruídas faça
6	Selecione randomicamente uma tarefa j em Π
7	Transfira j de Π para Ω
8	fim
9	Desloque todas as tarefas em Π para serem processadas o mais cedo possível
10	Atualize o custo da solução corrente
11	fim
12	retorna s
13	Fim

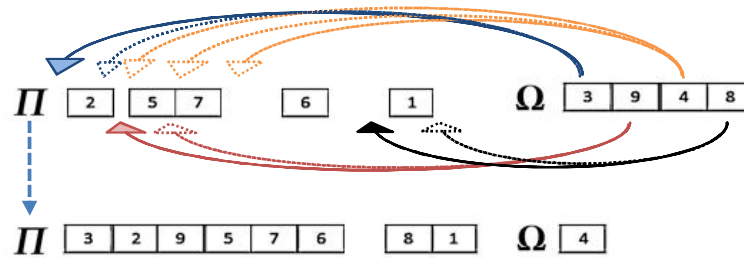
Fonte: Próprio autor.

3.1.3 Fase de construção

A fase de construção é dividida em dois processos. Inicialmente será aplicada uma sub-rotina que insere elementos de Ω em Π . Assim que nenhum elemento possa mais ser inserido, uma segunda sub-rotina, com o objetivo de aprimorar a solução corrente encontrada, buscará possibilidades de permutação entre as tarefas de ambos os conjuntos, e caso alguma permutação resulte numa função objetivo de menor valor, a solução corrente será atualizada.

3.1.3.1 Procedimento inserir

O procedimento inserir busca elementos em Ω aleatoriamente e calcula quais as possíveis posições para serem inseridos em Π . Caso exista mais de uma posição possível, o algoritmo irá inserir a tarefa na posição que resulte numa menor função objetivo. Após o momento que todos os elementos de Ω tiverem sido selecionados para uma tentativa de inserção, é inicializado o processo de permutação. A figura 3.3 a seguir ilustra o funcionamento deste procedimento com um exemplo numérico.

Figura 3.3 Procedimento inserir

Fonte: Próprio autor.

Neste exemplo, as tarefas rejeitadas 3, 9 e 8 foram inseridos na sequência de tarefas processadas. Cada elemento tinha mais de uma possibilidade de inserção, mas a que resultava numa menor função objetivo foi escolhida. Possibilidades de inserção também são rejeitadas caso restrições de tempos e custos sejam quebradas com o deslocamento a direita das tarefas subsequentes a inserida em Π . A modelagem computacional da fase inserir é ilustrada no Algoritmo 3.3, e suas sub-rotinas detalhadas no APÊNDICE C.

Algoritmo 3.3 Estrutura geral do procedimento inserir

inserir (s)

```

1  Atualize a quantidade de tarefas no conjunto  $\Omega$ 
2  para cada tarefa em  $\Omega$  faça
3      Selecione randomicamente uma tarefa  $j$  em  $\Omega$ 
4      Verifique as possíveis posições de inserir  $j$  em  $\Pi$ 
5      Função objetivo da solução inserida = Função objetivo da solução corrente
6      para cada posição possível de inserção faça
7          Insira a tarefa na posição possível
8          se custo de setup da tarefa + custo de processamento < custo de rejeição então
9              Ajuste os tempos de todas as tarefas processadas
10             se nenhuma restrição for quebrada então
11                 Ajuste os custos das tarefas processadas
12                 se resultado nova solução < resultado solução inserida então
13                     resultado solução inserida = resultado nova solução
14                     fim
15             fim
16         fim
17         Restaure solução corrente armazenada antes da inserção
18     fim
19     Atualize solução corrente com valores da melhor solução inserida encontrada
20 fim
21 retorna s
22 Fim

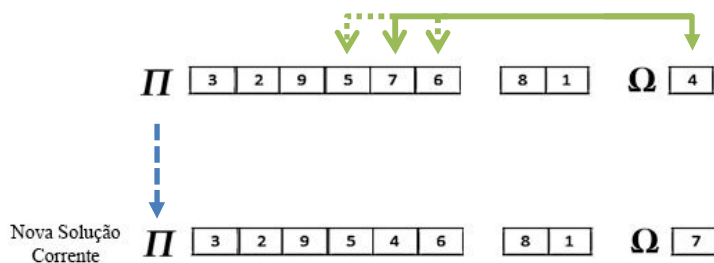
```

Fonte: Próprio autor.

3.1.3.2 Procedimento permutar

Buscando aprimorar a solução corrente definida pelo processo de inserção de elementos, foi desenvolvida um procedimento de permutação que busca uma menor função objetivo. A figura 3.4 a seguir ilustra o seu funcionamento.

Figura 3.4 Procedimento permutar



Fonte: Próprio autor.

Inicialmente o algoritmo escolherá aleatoriamente um elemento de Ω , e a partir de uma sub-rotina de verificação, será definido quais as possíveis tarefas a serem permutadas em Π sem que nenhuma restrição seja quebrada. Se alguma dessas permutações resultar numa função objetivo menor, a solução corrente será atualizada. O Algoritmo 3.4 a seguir ilustra a estrutura do procedimento permutar, e suas sub-rotinas detalhadas no APÊNDICE D.

Algoritmo 3.4 Estrutura geral da sub-rotina permutar

permutar (s)	
1	Atualiza a quantidade de tarefas no conjunto Ω
2	para cada tarefa em Ω faça
3	Selecione randomicamente uma tarefa j em Ω
4	Verifique as possíveis tarefas para permutar j em Π
5	Função objetivo da solução permutada = Função objetivo da solução corrente
6	para cada tarefa possível de permutação em Π faça
7	Permute j com a tarefa escolhida de Π
8	se custo de setup da tarefa + custo de processamento < custo de rejeição então
9	Ajuste os tempos de todas as tarefas processadas
10	se nenhuma restrição for quebrada então
11	Ajuste os custos das tarefas processadas
12	se resultado da nova solução < resultado da solução permutada então
13	resultado solução permutada = resultado nova solução
14	fim
15	fim
16	fim
17	Restaure solução corrente armazenada antes da permutação
18	fim
19	Atualize solução corrente com valores da melhor solução permutada encontrada
20	fim
21	retorna s
22	Fim

Fonte: Próprio autor.

3.1.4 Critérios de aceitação e de parada

Após a fase de construção, o algoritmo pode iniciar uma nova iteração destruindo a solução corrente ou destruindo a melhor solução candidata encontrada durante toda a execução. Este critério de aceitação deve ser definido antes da execução do algoritmo. Utilizar a solução candidata pode fazer com que se encontre um melhor resultado em um menor tempo, no entanto, em alguns casos, também pode conduzir a situações de estagnação de possibilidades devido a insuficiência de diversificação de soluções.

O algoritmo se encerra caso um dos critérios de parada seja satisfeito, sendo: o tempo de execução ultrapassar o tempo máximo estipulado, ou o algoritmo obter uma solução com resultado igual ou de menor valor à definida antes da execução.

4 APRESENTAÇÃO E DISCUSSÃO DOS RESULTADOS

O algoritmo foi desenvolvido em VBA no Microsoft Excel 2016 32 *bits*, e executado em um intel i7 2.3 GHz quadcore com 8 Gb de memória.

Os resultados obtidos foram comparados aos encontrados pelos 6 algoritmos e a uma programação linear (PL) desenvolvidos por Thevenin, Zufferey e Widmer (2015), implementados em C++ e executados por estes autores em um intel i7 2.93 GHz quadcore com 8 GB de memória.

Em cada uma das 44 instâncias, cada algoritmo foi executado cinco vezes, sendo definido como critério de parada, em segundos, $T=n*30$ (n é o número de tarefas). Para os resultados encontrados, o erro relativo da solução definido como $E_R = 100 * \frac{\text{Resultado Encontrado} - \text{Resultado ótimo}}{\text{Resultado ótimo}}$, foi calculado. O resultado ótimo é dado a partir do melhor valor da função objetivo conhecida da instância. Visando maior confiabilidade dos dados obtidos, foram desenvolvidas interfaces para execuções automatizadas e análise das replicações, ilustradas no APÊNDICE E.

Para cada algoritmo, a média do erro relativo de todas as execuções de cada uma das instâncias é apresentada, no formato: Média (Min – Max), com exceção da PL, a qual foi testada uma vez por instância, durante uma hora, sendo relatado o erro relativo.

Também foram comparados os tempos de execução do algoritmo proposto com a busca Tabu desenvolvida por Thevenin, Zufferey e Widmer (2015). Ao encontrar um resultado menor ou igual aos limites superiores encontrados em testes de 30 minutos por instância por Baptiste e Le Pape (2005) utilizando PL, a execução é interrompida e o tempo de execução calculado. Os resultados estão apresentados nas tabelas 4.1, 4.2 e 4.3.

No quadro 4.1 a seguir são apresentados os parâmetros de inicialização utilizados em cada instância, definidos com base em testes preliminares. Duas tarefas são destruídas por iteração em todas as execuções, com exceção das 4 maiores instâncias. Apesar de possuírem 500 tarefas, as 4 maiores instâncias possuem muitas tarefas idênticas, e destruir 50 tarefas por iteração e não utilizar o procedimento de permutação em duas delas resultou em menores tempos de processamento. Em relação ao critério de aceitação, foram obtidos melhores resultados destruindo a solução corrente em todos os testes, com exceção das instâncias de 90 tarefas, sendo elas: NCOS_41, NCOS_41a e STC_NCOS_41a. Por possuírem uma maior variabilidade de tarefas distintas, destruir a solução candidata resultou em menores tempos de processamento.

Quadro 4.1 Parâmetros de inicialização do algoritmo *Iterated Greedy*

Instâncias	d	Permutação	Critério de aceitação	Instâncias	d	Permutação	Critério de aceitação	Instâncias	d	Permutação	Critério de aceitação
NCOS_01	2	Sim	Sol. Corrente	NCOS_13a	2	Sim	Sol. Corrente	STC_NCOS_01	2	Sim	Sol. Corrente
NCOS_01a	2	Sim	Sol. Corrente	NCOS_14	2	Sim	Sol. Corrente	STC_NCOS_01a	2	Sim	Sol. Corrente
NCOS_02	2	Sim	Sol. Corrente	NCOS_14a	2	Sim	Sol. Corrente	STC_NCOS_15	2	Sim	Sol. Corrente
NCOS_02a	2	Sim	Sol. Corrente	NCOS_15	2	Sim	Sol. Corrente	STC_NCOS_15a	2	Sim	Sol. Corrente
NCOS_03	2	Sim	Sol. Corrente	NCOS_15a	2	Sim	Sol. Corrente	STC_NCOS_31	2	Sim	Sol. Corrente
NCOS_03a	2	Sim	Sol. Corrente	NCOS_31	2	Sim	Sol. Corrente	STC_NCOS_31a	2	Sim	Sol. Corrente
NCOS_04	2	Sim	Sol. Corrente	NCOS_31a	2	Sim	Sol. Corrente	STC_NCOS_32	2	Sim	Sol. Corrente
NCOS_04a	2	Sim	Sol. Corrente	NCOS_32	2	Sim	Sol. Corrente	STC_NCOS_32a	2	Sim	Sol. Corrente
NCOS_05	2	Sim	Sol. Corrente	NCOS_32a	2	Sim	Sol. Corrente	STC_NCOS_41	2	Sim	Sol. Corrente
NCOS_05a	2	Sim	Sol. Corrente	NCOS_41	2	Sim	Sol. Candidata	STC_NCOS_41a	2	Sim	Sol. Candidata
NCOS_11	2	Sim	Sol. Corrente	NCOS_41a	2	Sim	Sol. Candidata	STC_NCOS_51	2	Sim	Sol. Corrente
NCOS_11a	2	Sim	Sol. Corrente	NCOS_51	2	Sim	Sol. Corrente	STC_NCOS_51a	2	Sim	Sol. Corrente
NCOS_12	2	Sim	Sol. Corrente	NCOS_51a	2	Sim	Sol. Corrente	STC_NCOS_61	50	Sim	Sol. Corrente
NCOS_12a	2	Sim	Sol. Corrente	NCOS_61	50	Não	Sol. Corrente	STC_NCOS_61a	50	Sim	Sol. Corrente
NCOS_13	2	Sim	Sol. Corrente	NCOS_61a	50	Não	Sol. Corrente				

Fonte: Próprio autor.

Nos testes comparativos de erro relativo, o algoritmo mostrou um desempenho similar aos algoritmos mais eficientes de Thevenin, Zufferey e Widmer (2015). Das 44 instâncias testadas, em 36 a solução ótima foi encontrada (representadas por erro relativo mínimo = 0), e em duas instâncias o algoritmo proposto encontrou resultados superiores aos ótimos conhecidos (indicadas em azul nas tabelas). Nas demais instâncias, foram encontrados resultados próximos aos ótimos conhecidos. Os parâmetros dados pela instância foram representados exemplificativamente no APÊNDICE F e a interface dos resultados obtidos está detalhada no APÊNDICE G.

Tabela 4.1 Comparação entre os erros relativos encontrados nas instâncias desconsiderando Setup

Instâncias	Melhor	Tarefas	LP	Greedy	Descent	Tabu	TabuDiv	AMA ^{Mem}	AMA ^{SP}	Iterated Greedy
NCOS_01	800	8	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_01a	800	8	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_02	2570	10	0	12,8 (12,8-12,8)	12,8 (12,8-12,8)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_02a	1210	10	0	0,8 (0,8-0,8)	0,8 (0,8-0,8)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_03	6460	10	0	11,1 (11,1-11,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_03a	1690	10	0	3 (3-3)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_04	1011	10	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_04a	1008	10	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_05	1500	15	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_05a	1500	15	0	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_11	2022	20	0	6,1 (6,1-6,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_11a	2006	20	0,95	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_12	6844	24	2,73	10 (10-10)	10 (10-10)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_12a	4270	24	1,17	11,1 (11,1-11,1)	11,1 (11,1-11,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_13	3912	24	4,81	18,6 (18,6-18,6)	18,6 (18,6-18,6)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_13a	3441	24	17,58	9,6 (9,6-9,6)	9,6 (9,6-9,6)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_14	6990	25	51,65	1,7 (1,7-1,7)	1,7 (1,7-1,7)	0,3 (0-1,7)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_14a	3195	25	15,81	1,1 (1,1-1,1)	0,6 (0-0,9)	0,1 (0-0,6)	0,1 (0-0,6)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_15	3052	30	0,33	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_15a	3035	30	0,56	0,5 (0,5-0,5)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_31	9510	75	156,68	0 (0-0)	0 (0-0)	0,3 (0-0,8)	0,4 (0-1,1)	4,9 (4,4-5,5)	1,1 (0,7-1,9)	-0,1 (-0,5-0)
NCOS_31a	8715	75	95,75	4,5 (4,4-4,8)	4,7 (4,4-4,8)	0,4 (0,3-0,6)	0,7 (0,5-0,9)	0,2 (0-0,3)	0,9 (0,7-1,1)	0,1 (0-0,3)
NCOS_32	17310	75	313,06	4,2 (4,2-4,2)	4,2 (4,2-4,2)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)
NCOS_32a	14720	75	281,86	1,4 (1,4-1,4)	1,4 (1,4-1,4)	0,2 (0-0,3)	0,3 (0-0,3)	0,3 (0,3-0,3)	0,2 (0-0,3)	0 (0-0)
NCOS_41	13484	90	273,75	22,2 (22-22,7)	22,1 (20,9-23,5)	0,2 (0-0,4)	0,2 (0-0,3)	0,1 (0,1-0,2)	0,1 (0-0,2)	0,3 (0,1-0,3)
NCOS_41a	10539	90	44,23	8,2 (8-8,4)	10,3 (9,5-10,7)	0,1 (0,1-0,2)	0,1 (0-0,2)	0,1 (0-0,1)	0 (0-0,1)	0,3 (0,1-0,5)
NCOS_51	36170	200	*	5,8 (5,8-5,8)	4,1 (4,1-4,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0,1)
NCOS_51a	36170	200	*	6,1 (6,1-6,1)	4,1 (4,1-4,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0,036 (0-0,09)
NCOS_61	1269365	500	*	0,2 (0,2-0,2)	0,2 (0,2-0,2)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0,009 (0,004 - 0,01)
NCOS_61a	1485232	500	*	0,1 (0,1-0,1)	0,1 (0,1-0,1)	0 (0-0)	0 (0-0)	0 (0-0)	0 (0-0)	0,012 (0,006 - 0,02)
Média				4,6 (4,6-4,7)	3,9 (3,8-4)	0,1 (0-0,2)	0,1 (0-0,1)	0,2 (0,2-0,2)	0,1 (0-0,1)	0,02 (0-0,04)

Fonte: Adaptado de Thevenin, Zufferey e Widmer (2015).

Tabela 4.2 Comparação entre os erros relativos encontrados nas instâncias considerando Setup

Instâncias	Melhor	Tarefas	LP	Greedy	Descent	Tabu	TabuDiv	AMA ^{Mem}	AMA ^{SP}	Iterated Greedy
STC_NCOS_01	700	8	0	5,7 (5,7–5,7)	5,7 (5,7–5,7)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)
STC_NCOS_01a	610	8	0	1,6 (1,6–1,6)	1,6 (1,6–1,6)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)
STC_NCOS_15	17611	30	16,03	0,2 (0,2–0,2)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)
STC_NCOS_15a	5584	30	0,38	0,1 (0,1–0,1)	0 (0–0)	0 (0–0)	0 (0–0)	1,5 (0,2–5,1)	0 (0–0)	0 (0–0)
STC_NCOS_31	6615	75	88,13	15,3 (15,1–15,4)	3,8 (3,8–3,8)	4,5 (3,8–7,6)	4,5 (3,8–7,6)	3 (0–3,8)	4,8 (3,8–7,6)	0 (0–0)
STC_NCOS_31a	7590	75	216,6	25,8 (25,6–26)	3,3 (3,3–3,3)	4,7 (3,3–6,6)	3,3 (3,3–3,3)	1,8 (0–3,3)	6,6 (4,2–8,9)	0 (0–0)
STC_NCOS_32	24068	75	70,57	4,7 (4,7–4,7)	2,8 (2,2–3)	0,3 (0–0,8)	0,9 (0,3–1,9)	1,6 (1,3–1,9)	1,3 (0,2–1,9)	0 (-0,1–0)
STC_NCOS_32a	16798	75	174,81	3,9 (3,9–3,9)	0 (0–0)	0,5 (0–0,7)	0 (0–0)	1,2 (0,8–2,1)	0,4 (0,2–0,8)	0 (0–0)
STC_NCOS_41	43201	90	324,99	12,8 (12,8–12,9)	3,4 (3,4–3,4)	3,4 (3,3–3,4)	1,8 (0–3,4)	2,2 (0,2–3,5)	0,1 (0,1–0,1)	1,3 (0–3,3)
STC_NCOS_41a	18579	90	37,39	7,1 (7–7,1)	2,8 (2,8–2,8)	2,9 (2,8–3,4)	0,1 (0–0,4)	2,7 (2,4–3)	0,1 (0,1–0,1)	2,1 (0,038–4,2)
STC_NCOS_51	139675	200	*	74,7 (74,7–74,7)	74,7 (74,7–74,7)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	21,2 (0–45)
STC_NCOS_51a	148230	200	*	67,2 (67,2–67,2)	67,2 (67,2–67,2)	42,3 (42,3–42,3)	5,9 (2–21,3)	0 (0–0)	0,8 (0–2)	21,5 (6,6–43,7)
STC_NCOS_61	1495045	500	*	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0,1)	0 (0–0)
STC_NCOS_61a	1814605	500	*	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)	0 (0–0)
Média				15,6 (15,6–15,7)	11,8 (11,8–11,8)	4,2 (4–4,6)	1,2 (0,7–2,7)	1 (0,3–1,6)	1 (0,3–1,6)	3,3 (0,5–6,9)

Fonte: Adaptado de Thevenin, Zufferey e Widmer (2015).

Tabela 4.3 Tempo necessário para os algoritmos Tabu e IG alcançarem os limites superiores obtidos por Baptiste e Le Pape (2005)

Instâncias	Tabu (s)	IG (s)	Instâncias	Tabu (s)	IG (s)	Instâncias	Tabu (s)	IG (s)
NCOS_01	0,01	0,004	NCOS_13a	0,01	0,046	STC_NCOS_01	0,01	0,009
NCOS_01a	0,01	0,004	NCOS_14	0,01	0,105	STC_NCOS_01a	0,01	0,009
NCOS_02	0,01	0,009	NCOS_14a	13,53	0,028	STC_NCOS_15	0,01	0,046
NCOS_02a	0,01	0,008	NCOS_15	0,01	0,092	STC_NCOS_15a	0,01	0,044
NCOS_03	0,04	0,008	NCOS_15a	0,01	0,083	STC_NCOS_31	–	9,324
NCOS_03a	0,01	0,007	NCOS_31	0,01	0,297	STC_NCOS_31a	–	9,879
NCOS_04	0,01	0,011	NCOS_31a	0,01	0,286	STC_NCOS_32	0,01	1,572
NCOS_04a	0,01	0,01	NCOS_32	0,01	6,907	STC_NCOS_32a	66,99	10,254
NCOS_05	0,01	0,018	NCOS_32a	0,01	9,087	STC_NCOS_41	0,01	0,622
NCOS_05a	0,01	0,018	NCOS_41	0,01	0,939	STC_NCOS_41a	4,12	0,785
NCOS_11	0,1	0,081	NCOS_41a	4,14	0,931	STC_NCOS_51	0,03	5,634
NCOS_11a	0,01	0,062	NCOS_51	0,06	234,5	STC_NCOS_51a	0,03	4,462
NCOS_12	3,97	0,056	NCOS_51a	0,1	222,0	STC_NCOS_61	0,63	2,418,6
NCOS_12a	3,89	0,047	NCOS_61	0,72	1.841,4	STC_NCOS_61a	0,64	3.125,4
NCOS_13	0,01	0,048	NCOS_61a	0,54	1.216,4			

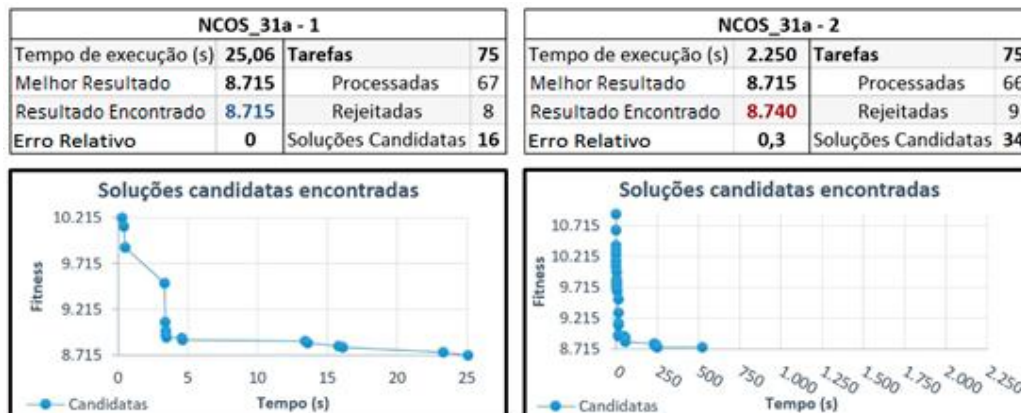
Fonte: Adaptado de Thevenin, Zufferey e Widmer (2015).

Em comparações obtidas por Rahim, Nayan e Hajiman (2006), um código em C++ é executado mais rapidamente ao desenvolvido em Visual Basic. Mesmo com esta limitação, o algoritmo proposto mostrou tempos competitivos em relação a busca Tabu, alcançando resultados iguais ou melhores aos limites superiores em poucos segundos para instâncias de até 200 tarefas.

Nos testes de erro relativo onde não foram obtidas a solução ótima, o principal problema não foi o tempo de processamento, mas sim a estagnação de possibilidades em encontrar melhores soluções.

A interface de análise das soluções candidatas encontradas durante uma execução está representada exemplificativamente no APÊNDICE H. A figura 4.1 apresenta as soluções candidatas encontradas em duas execuções na instância NCOS_31a. Na primeira, a solução ótima foi encontrada em 25 segundos. A segunda foi executada por 2.250 segundos e obtido erro relativo de 0,3. No intervalo de 521 até 2.250 segundos, correspondente a 76,8% do tempo total de execução, o algoritmo não foi capaz de encontrar uma melhor solução candidata.

Figura 4.1 Comparação das soluções candidatas encontradas em duas execuções da instância NCOS_31a



Fonte: Próprio autor.

Em trabalhos futuros, para evitar soluções candidatas estagnadas, pode-se implementar uma sub-rotina na fase de destruição que defina dinamicamente uma quantidade de tarefas a ser destruída ao longo da execução. Ao destruir um maior número de tarefas da solução corrente, ocorre o aumento de diversificação de possibilidades para que novas soluções sejam encontradas. Definido um novo caminho, deve-se diminuir a quantidade de tarefas destruídas, aumentando a sensibilidade do algoritmo de explorar o espaço de novas soluções e encontrar a ótima mais rapidamente.

Para medir intervalos de tempos, o algoritmo utiliza duas funções da API do Windows, sendo elas: *QueryPerformanceCounter()* e *QueryPerformanceFrequency()*. Estas funções fornecem acesso a contadores de alto desempenho implementados diretamente no hardware do processador, atingindo precisão máxima de até 1,8 μ s (CHENG et al., 2010). Verificou-se que estas funções possuem mínimo impacto no tempo de execução do algoritmo.

Os gráficos da figura 4.2 ilustram a distribuição do tempo de utilização das sub-rotinas empregadas na execução do algoritmo para todos os tamanhos de instância. O APÊNDICE I ilustra exemplificativamente a interface dos tempos obtidos para cada instância. Analisando os tempos das fases do algoritmo proposto, o procedimento inserir se torna o gargalo do algoritmo em problemas com mais de 10 tarefas. O procedimento permutar é o segundo com maior tempo de utilização, variando de acordo com o tamanho do conjunto de tarefas rejeitadas e a quantidade de iterações.

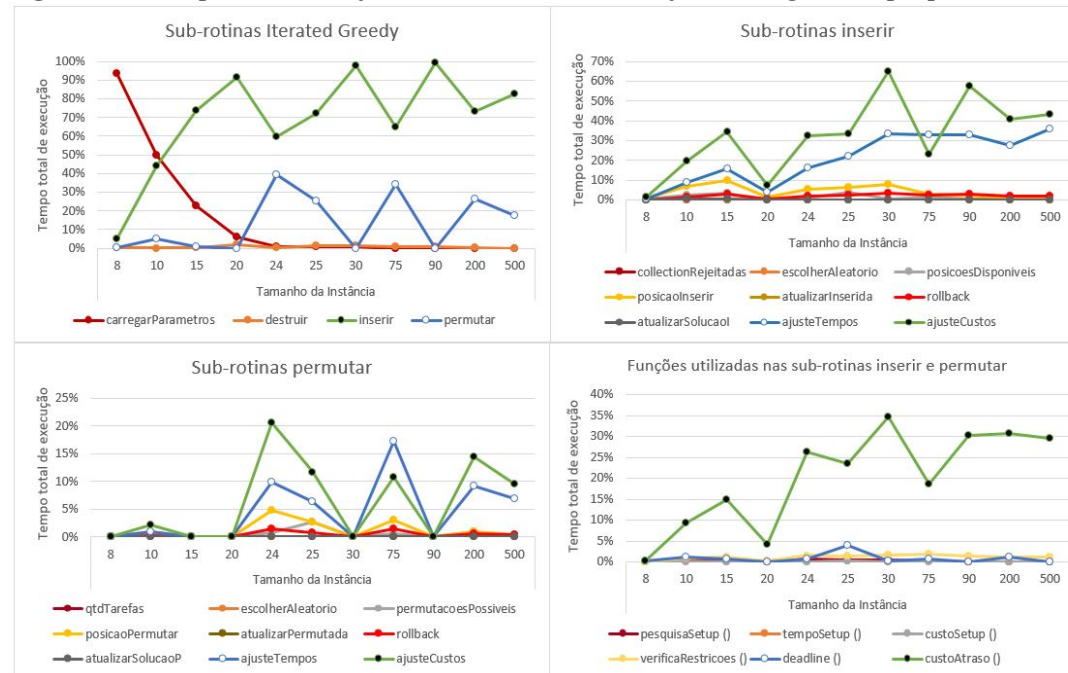
Em ambas as fases, as sub-rotinas *ajusteTempo* e *ajusteCustos* possuem a maior utilização de processamento computacional. Para cada tentativa de inserção ou

permutação, ao ocorrer deslocamento de posições das tarefas, o algoritmo recalcula os tempos e custos das tarefas processadas na solução. Ambas as sub-rotinas são utilizadas para realizar estes ajustes em todas as tarefas posteriores a tarefa inserida. A função `custoAtraso()` calcula o valor do custo da penalidade por atrasos da tarefa, sua utilização corresponde à metade do tempo de utilização da sub-rotina `ajusteCustos`, onde é chamada. Durante uma execução, ao modificar uma determinada tarefa da sequência, nem todas as tarefas posteriores terão os tempos ou custos alterados. Em trabalhos futuros, para otimizar as sub-rotinas gargalos para determinarem com exatidão quais tarefas precisarão ter seus tempos e custos recalculados, deve-se considerar os intervalos de tempo “ociosos” da sequência. Ao inserir ou permutar uma tarefa, ocorre o deslocamento de posição das tarefas subsequentes. Caso esse deslocamento seja menor ou igual a um período de ociosidade subsequente, a última tarefa com tempos e custos alterados da sequência será a última tarefa antes do período de ociosidade.

O algoritmo proposto aprimorou a formulação matemática para a data limite de entrega $\bar{d}_j$. Para Thevenin, Zufferey e Widmer (2015), $\bar{d}_j$ é calculada considerando apenas o custo de abandono u_j , ou seja, uma tarefa deverá ser rejeitada apenas quando o custo de atraso for superior ao custo de abandono. O algoritmo proposto considera que todos os custos da tarefa devam ser considerados no cálculo de $\bar{d}_j$, ou seja, uma tarefa deve ser rejeitada no momento em que a soma dos custos de setup, de processamento e de atraso forem superiores ao custo de abandono. Como o custo de setup varia de acordo com a posição da tarefa no sequenciamento, $\bar{d}_j$ torna-se dinâmico, melhorando as soluções encontradas. A data limite de entrega $\bar{d}_j$ dinâmica foi definida como:

$$\bar{d}_j = (\text{Min}(\text{END}_{MAX}; \text{Max}(d_j; d_j + \left(\frac{(\text{custo de rejeição} - \text{Custo de Setup} - \text{Custo de processamento})}{\text{TARDINESS}_{COST}} \right)))$$

Esta melhoria foi implementada como uma função chamada `deadline()`, e como indicado na figura 4.2, possui baixo processamento computacional.

Figura 4.2 Tempos de execuções das sub-rotinas e funções do algoritmo proposto

Fonte: Próprio autor.

Tabela 4.4 Resultados obtidos dos testes realizados para gerar os gráficos de tempos

	NCOS_01	NCOS_03	NCOS_05	NCOS_11	NCOS_12	NCOS_14	NCOS_15	NCOS_31	NCOS_41	NCOS_51	NCOS_61
Tarefas	8	10	15	20	24	25	30	75	90	200	500
Tempo de Execução (s)	0,005	0,01	0,02	0,15	1,6	0,6	1,17	28,5	2.700,02	1.739,28	15.100,84
Tarefas processadas	25%	90%	100%	100%	75%	92%	100%	99%	100%	97%	98%
Tarefas Rejeitadas	75%	10%	0%	0%	25%	8%	0%	1%	0%	3%	2%

Fonte: Próprio autor.

5 CONCLUSÃO

Durante esse trabalho, foi possível a obtenção de um algoritmo baseado na técnica *Iterated Greedy* que resolvesse problemas de *scheduling* em máquina única com rejeição e penalidade de atraso.

O algoritmo desenvolvido foi testado em problemas inspirados na manufatura, variando de 8 a 500 tarefas, apresentados por Nuijten et al (2003). Os resultados obtidos foram comparados aos encontrados por Baptiste e Le Pape (2005) e Thevenin, Zufferey e Widmer (2015). Percebeu-se que o algoritmo proposto é competitivo em relação as melhores metaheurísticas apresentadas na literatura para a resolução destes problemas. Das 44 instâncias testadas, 36 obtiveram resultados ótimos, 2 resultados melhores aos ótimos conhecidos e nas demais foram obtidos resultados próximos ao ótimo.

Pretende-se efetuar, em estudos futuros, a otimização das sub-rotinas gargalos e implementar uma função que defina quantas tarefas devem ser destruídas por iteração durante a execução do algoritmo, evitando a estagnação de possibilidades em encontrar melhores soluções.

Entende-se que este trabalho contribuiu para a literatura do *scheduling* ao propor formas inéditas de resolução para o problema $1 \mid r_j, s_{i,j}, \bar{d}_j \mid F_l(\sum f_j(C_j), \sum u_j, \sum c_{i,j})$. Adicionalmente, a metodologia e ferramentas adotadas para o desenvolvimento do algoritmo têm grande relevância na criação de aplicações práticas. Apesar do VBA ter uma performance inferior a outras linguagens mais utilizadas na literatura voltadas para otimização, esta característica só foi impactante para instâncias com mais de 200 tarefas. Por ser uma linguagem simples e de fácil acesso, aliado as diversas ferramentas de análise do Excel, o VBA permite a criação de sistemas flexíveis e versáteis com uma menor velocidade de desenvolvimento quando comparado a linguagens mais sofisticadas. Muitas vezes esse fator possui maior relevância a alguns minutos a mais de processamento ao buscar a solução de um problema.

REFERÊNCIAS

- ABDOLLAHPOUR, S. REZAEIAN, J. Minimizing makespan for flow shop scheduling problem with intermediate buffers by using hybrid approach of artificial immune system. **Applied Soft Computing**, v. 28, p. 44-56, 2015. Disponível em: <<http://migre.me/qrxM7>>. Acesso em: 22 maio 2015.
- BARTAL, Y. et al. Multiprocessor scheduling with rejection. **SIAM Journal on Discrete Mathematics**, v. 13, n 1, p. 64 – 78, 2006. Disponível em: <<http://migre.me/q9TtN>>. Acesso em: 04 abr. 2015.
- BAPTISTE, P.; LE PAPE, C. Scheduling a single machine to minimize a regular objective function under setup constraints. **Discrete Optimization**, V. 2, n.1, p.83-99, 2005. Disponível em: <<http://migre.me/shXkk>>. Acesso em: 15 maio 2015.
- BERTO, R. M. V. S.; NAKANO, D. N. A produção científica nos anais do encontro nacional de engenharia de produção: um levantamento de métodos e tipos de pesquisa. **Produção**, v. 9, n. 2. p. 65-75, 2000. Disponível em: <<http://migre.me/q8nRG>>. Acesso em: 23 maio 2015.
- BERTRAND, J.W.M.; FRANSOO, J.C. Modelling and simulation: operations management research methodologies using quantitative modeling. **International Journal of Operations & Production Management**, v.22, n.2, p.241-264, 2002. Disponível em: <<http://migre.me/q8ojv>>. Acesso em: 23 maio 2015.
- BLACKSTONE JUNIOR, J. H. **Apics dictionary**: the essential supply chain reference. 14 ed. Chicago: Apics, 2013. Disponível em: <<http://migre.me/qqRES>>. Acesso em: 19 abr. 2015.
- CESARET, B.; OĞUZ, C.; SALMAN, F.S. A tabu search algorithm for order acceptance and scheduling. **Computers e Operations Research**, v. 39, p. 1197-1205, 2012. Disponível em: <<http://migre.me/qwOHZ>>. Acesso em: 17 abr. 2015
- CHENG, F.; YIN, H.; ZHANG, Q.; ZHANG, D. Research of Accurate Software Timing Based on Windows. **International Conference on Challenges in Environmental Science and Computer Engineering**, p. 104 – 106, 2010. Disponível em: <<http://migre.me/seglu>>. Acesso em: 17 set. 2015.
- FERNANDES, F. C. F.; GODINHO FILHO, M. **Planejamento e controle da produção: dos fundamentos ao essencial**. São Paulo: Atlas, 2010. 296 p.

GRAHAM, R. L. et al. Optimization and approximation in deterministic sequencing and scheduling: a survey. **Anais de Matemática Discreta**, v. 5, p. 287 – 326, 1979. Disponível em: <<http://migre.me/qmNhe>>. Acesso em: 17 maio 2015.

HERRMANN, J. W. A History of production scheduling. In: HERRMANN, J.W. (Editor) **Handbook of production scheduling**. New York: Springer, 2006. 318p. Disponível em: <<http://migre.me/qqFd4>>. Acesso em: 14 abr. 2015.

HERRMANN, J. W. The perspective of Taylor, Gantt, and Johnson: how to improve production scheduling. **IJOQM**, v. 16, n. 3, p. 243 – 254, 2010. Disponível em: <<http://migre.me/qrJSf>>. Acesso em: 14 abr. 2015.

JOHNSON, S. M. Optimal two-and three-stage production schedules with setup times included. **Naval Research Logistics**, v. 1, issue 1, p. 61-68, 1954. Disponível em: <<http://migre.me/q9Twc>>. Acesso em: 20 abr. 2015.

LU, L.; NG, C.T.; ZHANG, L. Optimal algorithms for single-machine scheduling with rejection to minimize the makespan. **International Journal of Production Economics**, v. 130, issue 2, p. 153-158, 2011. Disponível em: <<http://migre.me/q9TBT>>. Acesso em: 18 maio 2015.

MACCARTHY, B. L. Organizational, systems and human issues in production planning, scheduling and control. In: HERRMANN, J. (Editor) **Handbook of Production Scheduling**. New York: Springer, 2006. (International Series in Operations Research and Management Science) Capítulo 3, p. 59–90. Disponível em: <<http://migre.me/qqLdk>>. Acesso em: 14 abr. 2015.

MARTINS, R. A. Abordagens quantitativa e qualitativa. In: MIGUEL, P.A. C. (Coordenador) **Metodologia de pesquisa em engenharia de produção e gestão de operações**. Rio de Janeiro: Elsevier, 2012. 280p. (Coleção ABEPRO) Capítulo 3, p. 47-63.

MORABITO, R.; PUREZA, V. Modelagem e simulação. In: MIGUEL, P.A. C. (Coordenador) **Metodologia de pesquisa em engenharia de produção e gestão de operações**. Rio de Janeiro: Elsevier, 2012. 280p. (Coleção ABEPRO) Capítulo 8, p. 169-198.

NAWAZ, M. ENSCORE JUNIOR, E. HAN, I. A heuristic algorithm for m-machine, n-job flow-shop sequencing problem. **Omega**, vol. 11, p. 91-95, 1983. Disponível em <<http://migre.me/qrxnW>>. Acesso em: 15 abr. 2015.

NUIJTEN, W. BOUSONVILLE, T. FOCACCI, F. GODARD, D. LE PAPE, C.
Towards an industrial manufacturing scheduling problem and test bed. Ilog S.A.,
 Gantilly, França, 2003. <<http://migre.me/seqiF>>. Acesso em: 18 ago. 2015.

PINEDO, M. **Scheduling: Theory, algorithms, and systems.** Berlin: Springer, 2012.
 Disponível em: <<http://migre.me/qdoiz>>. Acesso em: 22 maio 2015.

RAHIM, R. A. MOHD, N. RAHIMAN, H. F. Ultrasonic tomography system for
 liquid/gas flow: frame rate comparison between visual basic and visual c++
 programming. *Jurnal Teknologi*, 44(D) Jun 2006: 131–150. Disponível em:
 <<http://migre.me/segvH>> Acesso em: 02 nov. 2015.

RUIZ, R.; SERIFOGLU, S. F.; URLINGS, T. Modeling realistic hybrid flexible
 flowshop scheduling problems. **Computers e Operations Research**, v. 35, issue 4, p.
 1151 – 1175, 2008. Disponível em: <<http://migre.me/q9TMn>>. Acesso em: 15 maio
 2015.

RUIZ, R. STUTZLE, T. An iterated greedy algorithm for the flowshop problem with
 sequence dependent setup times. In: THE METAHEURISTIC INTERNATIONAL
 CONFERENCE, 6, 2005, AUSTRIA, **Anais...** Austria: MIC2005, p. 817-823.
 Disponível em: <<http://migre.me/qsC0P>>. Acesso em: 12 maio 2015.

RUIZ, R.; STUTZLE, T. An iterated greedy heuristic for the sequence dependent setup
 times flowshop problems with makespan and weighted tardiness objectives. **European
 Journal of Operational Research**, v. 187, n.3, p. 1143 – 1159, 2008. Disponível em:
 <<http://migre.me/q9TQL>>. Acesso em: 12 maio 2015.

SADEH, N. Micro – opportunistic scheduling: the micro – boss factory scheduler. In:
 ZWEBEN, M; FOX, M. S. (Editores) **Intelligent Scheduling**. São Francisco: Morgan
 Kaufman, 1994. p. 44. Capítulo 4. Disponível em: <<http://migre.me/qrlNy>>. Acesso
 em: 19 jun. 2015.

SHABTAY, D.; GASPAR, N.; KASPI, M. A survey on offline scheduling with
 rejection. **Journal of Scheduling**, v. 16, p. 3 – 28, 2013. Disponível em:
 <<http://migre.me/qwOR8>>. Acesso em: 17 abr. 2015.

SLOTNICK, S. A. Order acceptance and scheduling: a taxonomy and review.
European Journal of Operational Research, v. 212, p. 1-11, 2012. Disponível em:
 <<http://migre.me/qwP2n>>. Acesso em: 17 abr. 2015.

STUTZLE, T.; DORIGO, M. **Ant colony optimization**. Massachusetts: Asco Typesetters, 2004. 395 p. Disponível em: <<http://migre.me/qdmQ6>>. Acesso em: 2 maio 2015.

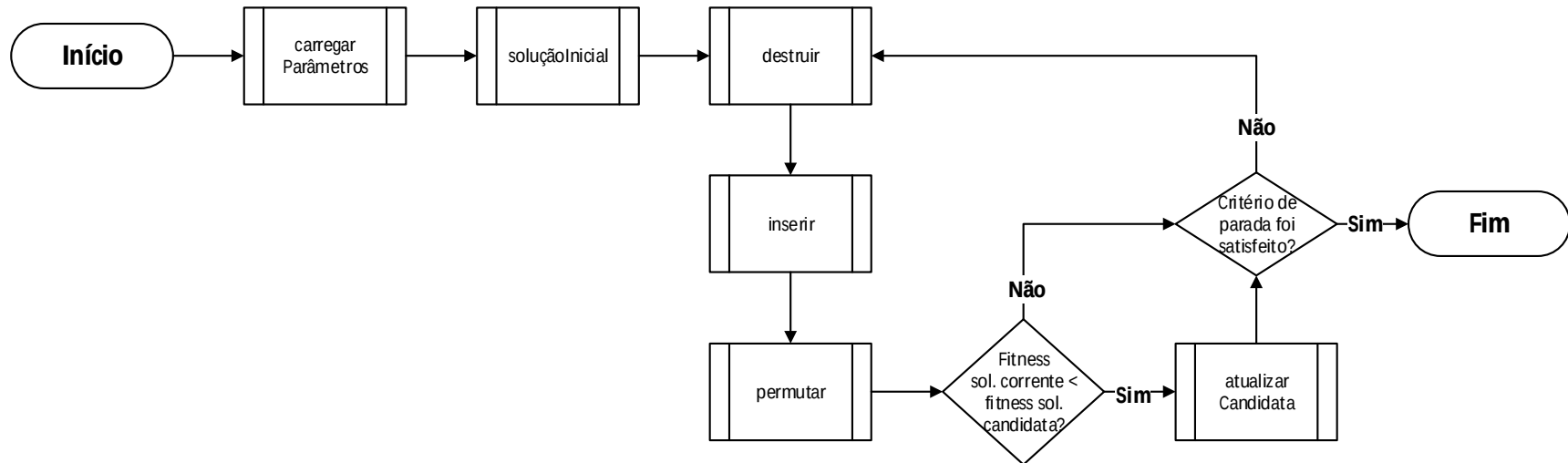
TERENCE, A, C, F.; ESCRIVÃO FILHO, E.; Abordagem quantitativa, qualitativa e a utilização de pesquisa – ação nos estudos organizacionais. IN: ENCONTRO NACIONAL DE ENGENHARIA DE PRODUÇÃO, 26, 2006, FORTALEZA. **Anais...** Fortaleza: ABEPRO, 2006. Disponível em: <<http://migre.me/q8nVa>>. Acesso em: 22 maio 2015.

THEVENIN S.; ZUFFEREY, N.; WIDMER, M. Metaheuristics for a scheduling problem with rejection and tardiness penalties. **Journal of Scheduling**, v.18, n. 1, p. 89-105, 2015. Disponível em: <<http://migre.me/q8nTw>>. Acesso em: 1 maio 2015

WIGHT, O. W. **Production and inventory management in the computer age**. Boston: Cbi Publishing Company, 1984. 296 p. Disponível em: <<http://migre.me/qsnHq>>. Acesso em: 14 abr. 2015.

ZUFFEREY, N. Metaheuristics: some principles for an efficient design. **Computer Technology an Application**, v. 3, p. 446-462, 2012. Disponível em: <<http://migre.me/qwsRz>> Acesso em: 15 jun. 2015.

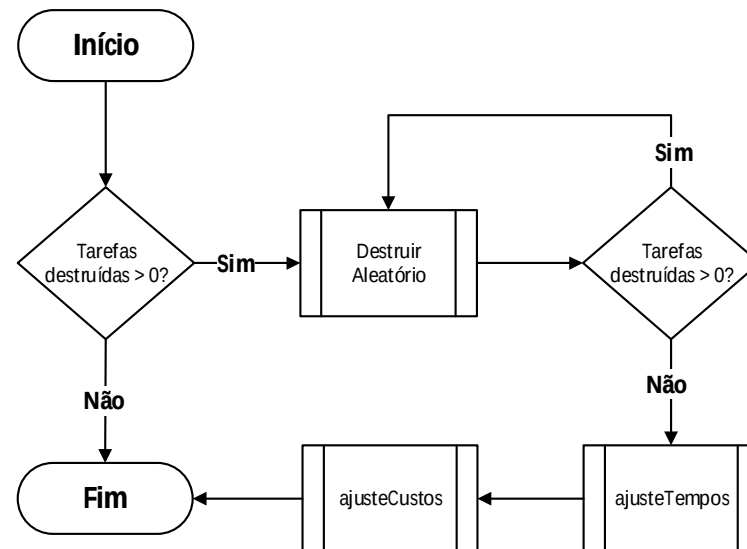
APÊNDICE A - Fluxograma das etapas do algoritmo desenvolvido



Quadro descritivo das etapas do algoritmo desenvolvido

Sub-rotina	Descrição
carregarParâmetros	Carrega os parâmetros da instância para as estruturas de dados na memória do algoritmo.
soluçãoInicial	Gera a solução inicial.
destruir	Rejeita aleatoriamente tarefas processadas.
inserir	Insere tarefas rejeitadas na solução corrente.
permutar	Permuta tarefas rejeitadas na solução corrente.
atualizarCandidata	Atualiza a solução candidata caso um menor resultado seja encontrado.

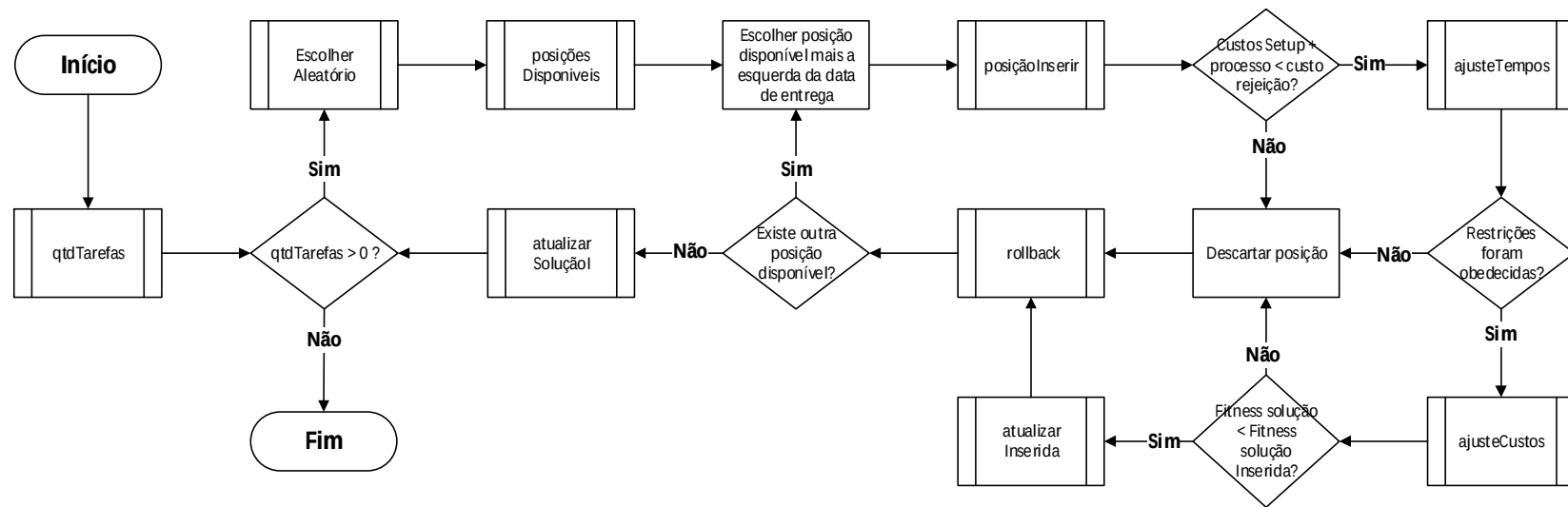
APÊNDICE B - Fluxograma das etapas da fase de destruição



Quadro descritivo das etapas da fase de destruição

Sub-rotina	Descrição
destruirAleatório	Remove aleatoriamente uma tarefa processada da solução corrente, e atualiza valores dependentes do setup da tarefa sucessora.
ajusteTempos	Ajusta os tempos de todas as tarefas processadas, deslocando-as para a esquerda.
ajusteCustos	Ajusta os custos de todas as tarefas.

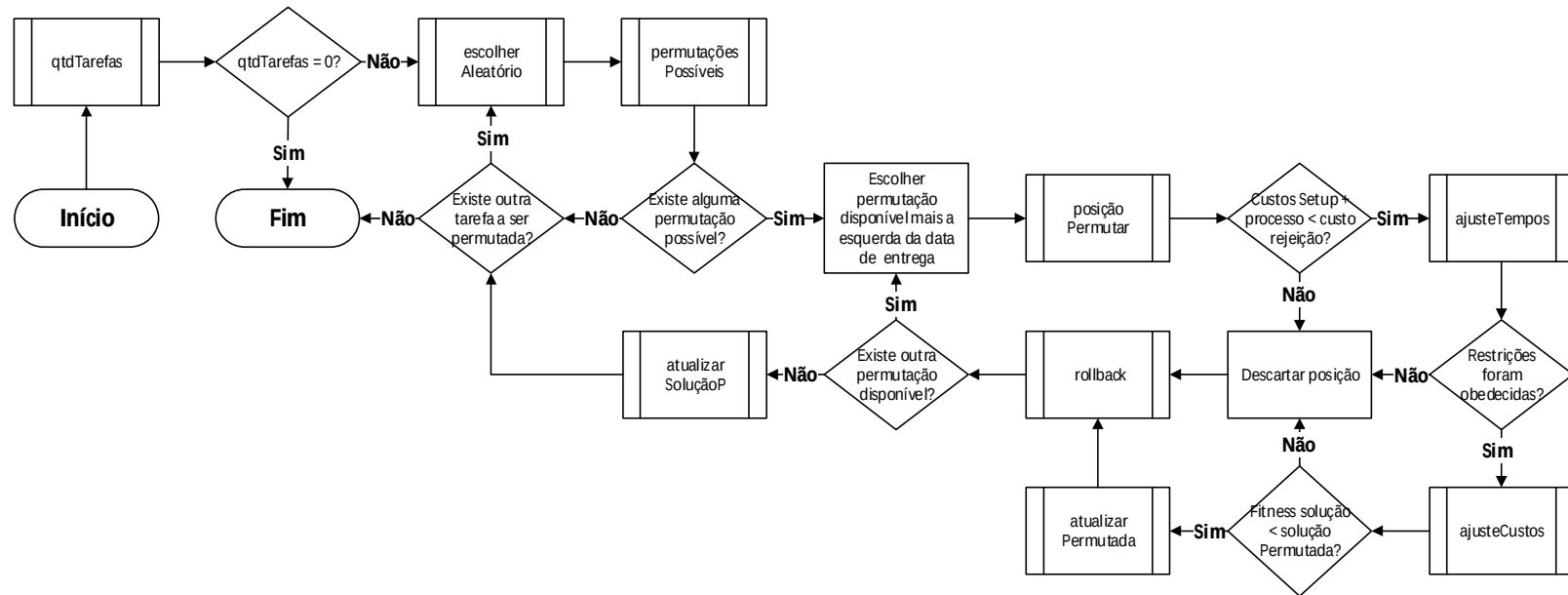
APÊNDICE C - Fluxograma das etapas do procedimento inserir



Quadro descritivo das etapas da fase de inserção

Sub-rotina	Descrição
qtdTarefas	Calcula a quantidade de tarefas que podem tentar ser inseridas.
escolherAleatório	Escolhe aleatoriamente uma tarefa que pode ser inserida.
posiçõesDisponíveis	Define o intervalo de possíveis posições de inserção. Faz backup da solução corrente, utilizado na rotina rollback.
posiçãoInserir	Ajusta variáveis da tarefa inserida e da tarefa sucessora.
ajusteTempos	Ajusta os tempos de todas as tarefas processadas.
ajusteCustos	Ajusta os custos de todas as tarefas.
atualizarInserida	Atualiza solução inserida.
rollback	Restaura solução corrente armazenada antes da inserção.
atualizarSoluçãoI	Atualiza solução corrente com valores da melhor solução inserida encontrada.

APÊNDICE D - Fluxograma das etapas do procedimento permutar



Quadro descritivo das etapas da fase de permutação

Sub-rotina	Descrição
qtdTarefas	Atualiza o conjunto das tarefas rejeitadas.
escolherAleatório	Escolhe aleatoriamente uma tarefa rejeitada.
permutaçõesPossíveis	Define o intervalo de possíveis posições de permutação. Faz backup da solução, utilizado na rotina rollback.
posiçãoPermutar	Ajusta variáveis da tarefa permutada e da tarefa sucessora.
ajusteTempos	Ajusta os tempos de todas as tarefas processadas.
ajusteCustos	Ajusta os custos de todas as tarefas.
atualizarPermutada	Atualiza solução permutada.
rollback	Restaura solução corrente armazenada antes da permutação.
atualizarSolucaoP	Atualiza solução com valores da melhor solução permutada encontrada.

APÊNDICE E - Interfaces de execução e análise de replicações

Iterated Greedy - Executar Instâncias em Lote

Replicações: **5** **Parâmetros iguais**

1 2 3 4 5

2 Tarefas destruídas

☒ Permutação

☐ Destruir Solução Candidata

☒ Destruir Solução Corrente

☒ Tempo Máximo (s) = $n * 30$

Inici **Fechar**

Instâncias

Marcar Todas Desmarcar Todas

☒ NCOS_01	☒ NCOS_11a	☒ NCOS_32	☒ STC_NCOS_15a
☒ NCOS_01a	☒ NCOS_12	☒ NCOS_32a	☒ STC_NCOS_31
☒ NCOS_02	☒ NCOS_12a	☒ NCOS_41	☒ STC_NCOS_31a
☒ NCOS_02a	☒ NCOS_13	☒ NCOS_41a	☒ STC_NCOS_32
☒ NCOS_03	☒ NCOS_13a	☒ NCOS_51	☒ STC_NCOS_32a
☒ NCOS_03a	☒ NCOS_14	☒ NCOS_51a	☒ STC_NCOS_41
☒ NCOS_04	☒ NCOS_14a	☒ NCOS_61	☒ STC_NCOS_41a
☒ NCOS_04a	☒ NCOS_15	☒ NCOS_61a	☒ STC_NCOS_51
☒ NCOS_05	☒ NCOS_15a	☒ STC_NCOS_01	☒ STC_NCOS_51a
☒ NCOS_05a	☒ NCOS_31	☒ STC_NCOS_01a	☒ STC_NCOS_61
☒ NCOS_11	☒ NCOS_31a	☒ STC_NCOS_15	☒ STC_NCOS_61a

Iterated Greedy - Analisar Replicações

Instância **Tarefas: 30** **Limpar Instância**

STC_NCOS_15 **Melhor Função Objetivo: 17611** **Limpar Todos**

Replicações

1 2 3 4 5

Parâmetros

	1	2	3	4	5
Permutação	Sim	Sim	Sim	Sim	Sim
Tarefas Destruidas	2	2	2	2	2
Solução Destruida	Corrente	Corrente	Corrente	Corrente	Corrente

Resultados

	1	2	3	4	5
Soluções Candidatas	14	17	9	18	18
Função Objetivo	17611	17611	17611	17611	17611
Tempo de Execução	2,773	2,110	0,615	1,839	5,122
Erro Relativo	0,0	0,0	0,0	0,0	0,0

Médias

Tempo Médio: 2,492 **Erro Relativo Médio: 0,0**

Fechar

APÊNDICE F - Representação exemplificativa dos parâmetros de uma instância

Parâmetros Setup			
Família Antecessora	Família Sucessora	Tempo Setup	Custo Setup
0	0	0	0
0	1	100	100
0	2	60	80
1	0	50	50
1	1	0	0
1	2	90	100
2	0	10	10
2	1	40	40
2	2	0	0

Parâmetros Instância	
Total Tarefas	30
Estado Inicial de Setup	0
Critério parada Função Objetivo	17.611
Critério parada Tempo	900
Tarefas Destruidas	2
Total de Famílias	3
Permutação ativada?	Sim
Destruir Solução	Corrente
END MAX	800

Parâmetros Tarefas							
Tarefa	Família	Data de Liberação	Data de Entrega	Tempo Processamento	Custo Processamento	Custo Rejeição	Tardiness Weight
0	0	14	58	1	100	200	10
1	2	17	59	1	100	200	10
2	2	90	131	1	100	200	8
3	1	83	158	1	100	200	3
4	2	70	79	2	100	400	3
5	0	7	53	4	100	800	7
6	1	6	33	5	100	1.000	10
7	2	65	70	5	100	1.000	9
8	1	14	78	5	100	1.000	5
9	2	81	124	5	100	1.000	10
10	1	45	134	5	100	1.000	8
11	0	93	135	6	100	1.200	9
12	0	13	42	7	100	1.400	6
13	1	54	64	7	100	1.400	3
14	0	16	69	7	100	1.400	3
15	1	55	99	7	100	1.400	8
16	1	56	106	7	100	1.400	5
17	1	96	145	7	100	1.400	5
18	1	74	155	7	100	1.400	8
19	1	62	123	8	100	1.600	9
20	2	48	58	9	100	1.800	8
21	1	34	116	9	100	1.800	4
22	1	68	127	9	100	1.800	7
23	1	49	147	9	100	1.800	4
24	1	41	51	10	100	2.000	6
25	0	49	64	10	100	2.000	9
26	1	38	76	10	100	2.000	6
27	0	71	84	10	100	2.000	7
28	1	15	114	10	100	2.000	8
29	1	87	135	10	100	2.000	6

APÊNDICE G - Representação de uma solução obtida pelo algoritmo proposto

Posição	Π	Ω
1	5	0
2	12	1
3	7	2
4	4	6
5	20	11
6	9	25
7	10	
8	15	
9	3	
10	18	
11	19	
12	8	
13	28	
14	22	
15	16	
16	17	
17	26	
18	29	
19	24	
20	21	
21	23	
22	13	
23	27	
24	14	

Tarefa	Processada?	Tempo Setup	Custo Setup	Data Limite	Start_Max	Início Setup	Início Processo	Fim Processo	Custo Atraso	Custo Total
0	Não	0	0	0	0	0	0	0	0	200
1	Não	0	0	0	0	0	0	0	0	200
2	Não	0	0	0	0	0	0	0	0	200
3	Sim	0	0	191	190	153	153	154	0	100
4	Sim	0	0	179	177	85	85	87	24	124
5	Sim	0	0	153	149	7	7	11	0	100
6	Não	0	0	0	0	0	0	0	0	1.000
7	Sim	60	80	161	156	20	80	85	135	315
8	Sim	0	0	258	253	169	169	174	480	580
9	Sim	0	0	214	209	96	96	101	0	100
10	Sim	40	40	242	237	101	141	146	96	236
11	Não	0	0	0	0	0	0	0	0	1.200
12	Sim	0	0	259	252	13	13	20	0	100
13	Sim	0	0	497	490	255	255	262	594	694
14	Sim	0	0	502	495	322	322	329	780	880
15	Sim	0	0	262	255	146	146	153	432	532
16	Sim	0	0	366	359	193	193	200	470	570
17	Sim	0	0	405	398	200	200	207	310	410
18	Sim	0	0	318	311	154	154	161	48	148
19	Sim	0	0	290	282	161	161	169	414	514
20	Sim	0	0	271	262	87	87	96	304	404
21	Sim	0	0	541	532	237	237	246	520	620
22	Sim	0	0	370	361	184	184	193	462	562
23	Sim	0	0	572	563	246	246	255	432	532
24	Sim	0	0	368	358	227	227	237	1.116	1.216
25	Não	0	0	0	0	0	0	0	0	2.000
26	Sim	0	0	393	383	207	207	217	846	946
27	Sim	50	50	348	338	262	312	322	1.666	1.816
28	Sim	0	0	352	342	174	174	184	560	660
29	Sim	0	0	452	442	217	217	227	552	652

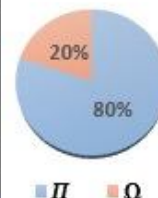
APÊNDICE H - Representação exemplificativa das soluções candidatas encontradas durante uma execução



Candidatas	Função Objetivo	Tempo (s)	Π	Ω	Iterações
1	18.212	0,0555	23	7	1
2	17.989	0,0763	26	4	2
3	17.988	0,0897	26	4	3
4	17.948	0,1146	26	4	5
5	17.829	0,1936	26	4	12
6	17.710	0,2049	26	4	13
7	17.698	0,2770	26	4	19
8	17.690	0,2876	26	4	20
9	17.635	0,3106	26	4	22
10	17.621	0,4718	25	5	35
11	17.614	0,5114	25	5	38
12	17.613	0,5556	25	5	41
13	17.612	1,1030	24	6	82
14	17.611	2,7733	24	6	204

APÊNDICE I - Representação exemplificativa dos tempos e execuções das sub-rotinas do algoritmo obtidos durante uma execução

STC_NCOS_15			
Tempo de execução (s)	2,77331	Tarefas	30
Melhor Resultado	17.611	Processadas	24
Resultado Encontrado	17.611	Rejeitadas	6
Erro Relativo	0	Tardiness Weight	Sim
Permutação	Sim	SETUP	Sim
Tarefas destruídas	2	Solução destruída	Corrente



Fluxogramas	Rotinas	Execuções	Tempo Total (s)	T Total (%)	T Médio (ms)
Iterated Greedy	carregarParametros	1	0,01075	0,39%	10,7483
	solucaoinicial	1	0,00026	0,01%	0,2579
	destruir	204	0,05026	1,81%	0,2464
	inserir	204	1,92096	69,27%	9,4165
	permutar	204	0,79016	28,49%	3,8733
	atualizarCandidata	14	0,00015	0,01%	0,0110
Destruir	destruirAleatorio	406	0,00446	0,16%	0,0110
	ajusteTemposd	203	0,01466	0,53%	0,0722
	ajusteCustosd	203	0,02937	1,06%	0,1447
Inserir	qtdTarefas	204	0,00085	0,03%	0,0042
	escolherAleatorio	1.389	0,00272	0,10%	0,0020
	posicoesDisponiveis	1.389	0,24502	8,83%	0,1764
	posicaoInserir	14.253	0,27910	10,06%	0,0196
	ajusteTempos	14.190	0,67184	24,23%	0,0473
	ajusteCustos	4.885	0,46419	16,74%	0,0950
	atualizarInserida	2.185	0,01766	0,64%	0,0081
	rollback	14.253	0,10668	3,85%	0,0075
	atualizarSolucaoI	1.389	0,01051	0,38%	0,0076
Permutar	qtdTarefas	204	0,00068	0,02%	0,0033
	escolherAleatorio	959	0,00186	0,07%	0,0019
	permutacoesPossiveis	959	0,21013	7,58%	0,2191
	posicaoPermutar	5.831	0,16257	5,86%	0,0279
	ajusteTempos	5.826	0,29357	10,59%	0,0504
	ajusteCustos	182	0,02560	0,92%	0,1406
	atualizarPermutada	6	0,00005	0,00%	0,0077
	rollback	5.831	0,04943	1,78%	0,0085
	atualizarSolucaoP	552	0,00419	0,15%	0,0076
Funções	pesquisaSetup ()	40.449	0,02674	0,96%	0,0007
	tempoSetup ()	42.487	0,02116	0,76%	0,0005
	custoSetup ()	37.559	0,01543	0,56%	0,0004
	deadline ()	37.559	0,27191	9,80%	0,0072
	custoAtraso ()	66.089	0,21479	7,74%	0,0032
	verificaRestricoes ()	20.016	0,07233	2,61%	0,0036